Vivekanand Education Society's Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai
Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Vanshika Ambwani** Of **D15A** semester **VI,** have successfully completed necessary experiments in the **MAD & PWA Lab** under my supervision in **VES Institute of Technology** during the academic year **2024-2025.**

Lab Assistant

Subject Teacher

Mrs. Kajal Joseph

Principal

Head of Department

Dr. Mrs. Shalu Chopra

Name of the Course : MAD & PWA Lab **Course Code :** ITL604

Year/Sem/Class : D15A/D15B **A.Y.:** 24-25

Faculty Incharge : Mrs. Kajal Joseph.

Lab Teachers : Mrs. Kajal Joseph.

Email : kajal.jewani@ves.ac.in

Programme Outcomes: The graduate will be able to:

PO1) Basic Engineering knowledge: An ability to apply the fundamental knowledge in mathematics, science and engineering to solve problems in Computer engineering.

PO2) Problem Analysis: Identify, formulate, research literature and analyze computer engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and computer engineering and sciences.

PO3) Design/ Development of Solutions: Design solutions for complex computer engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal and environmental considerations.

PO4) Conduct investigations of complex engineering problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of information to provide valid conclusions.

PO5) Modern Tool Usage: Create, select and apply appropriate techniques, resources and modern computer engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6) The Engineer and Society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to computer engineering practice.

PO7) Environment and Sustainability: Understand the impact of professional computer engineering solutions in societal and environmental contexts and demonstrate knowledge of and need for sustainable development.

PO8) Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of computer engineering practice.

PO9) Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

PO10) Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.

PO11) Project Management and Finance: Demonstrate knowledge and understanding of computer engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12) Life-long Learning: Recognize the need for and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

Program specific Outcomes

PSO1) An ability to manage and analyze data / information effectively for making better decisions.

PSO2) Demonstrate the ability to use state of the art technologies and tools including Free and Open Source Software (FOSS) tools in developing software.

Lab Objectives:

Sr. No.	Lab Objectives
The Lab experiments aims:	
1	Learn the basics of the Flutter framework.
2	Develop the App UI by incorporating widgets, layouts, gestures and animation
3	Create a production ready Flutter App by including files and firebase backend service.
4	Learn the Essential technologies, and Concepts of PWAs to get started as quickly and efficiently as possible
5	Develop responsive web applications by combining AJAX development techniques with the jQuery JavaScript library.
6	Understand how service workers operate and also learn to Test and Deploy PWA.

Lab Outcomes:

Sr. No.	Lab Outcomes	Cognitive levels of attainment as per Bloom's Taxonomy
On Completion of the course the learner/student should be able to:		
1	Understand cross platform mobile application development using Flutter framework	L1, L2
2	Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation	L3
3	Analyze and Build production ready Flutter App by incorporating backend services and deploying on Android / iOS	L3, L4
4	Understand various PWA frameworks and their requirements	L1, L2
5	Design and Develop a responsive User Interface by applying PWA Design techniques	L3
6	Develop and Analyse PWA Features and deploy it over app hosting solutions	L3, L4

Index

Sr. No	Experiment Title	LO	DOP	DOS	Grade
1.	To install and configure the Flutter Environment	LO1			
2.	To design Flutter UI by including common widgets.	LO2			
3.	To include icons, images, fonts in Flutter app	LO2			
4.	To create an interactive Form using form widget	LO2			
5.	To apply navigation, routing and gestures in Flutter App	LO2			
6.	To Connect Flutter UI with fireBase database	LO3			
7.	To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.	LO4			
8.	To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA	LO5			
9.	To implement Service worker events like fetch, sync and push for E-commerce PWA	LO5			
10.	To study and implement deployment of Ecommerce PWA to GitHub Pages.	LO5			
11.	To use google Lighthouse PWA Analysis Tool to test the PWA functioning.	LO6			
12.	Assignment-1	LO1,LO2 ,LO3			
13.	Assignment-2	LO4,LO5 ,LO6			

MAD & PWA Lab
Journal

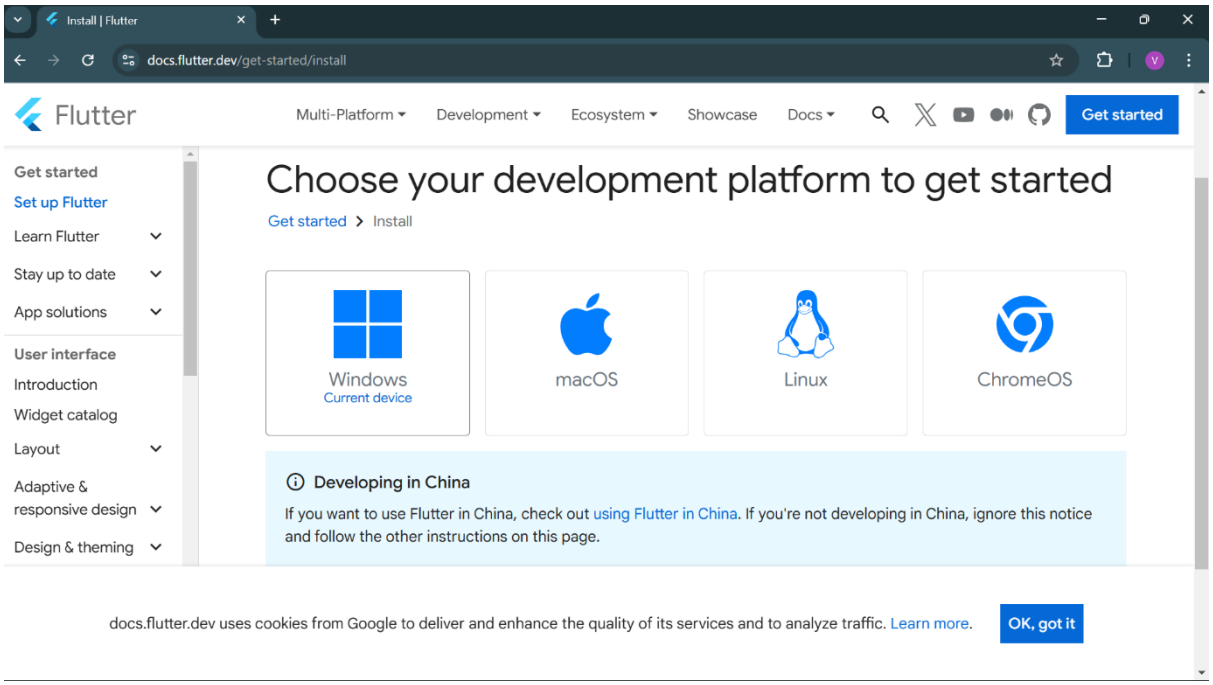
Experiment No.	01
Experiment Title.	To install and configure the Flutter Environment
Roll No.	02
Name	Vanshika Ambwani
Class	D15A/
Subject	MAD & PWA Lab
Lab Outcome	LO1: Understand cross platform mobile application development using Flutter framework
Grade:	

EXPERIMENT NO: - 01

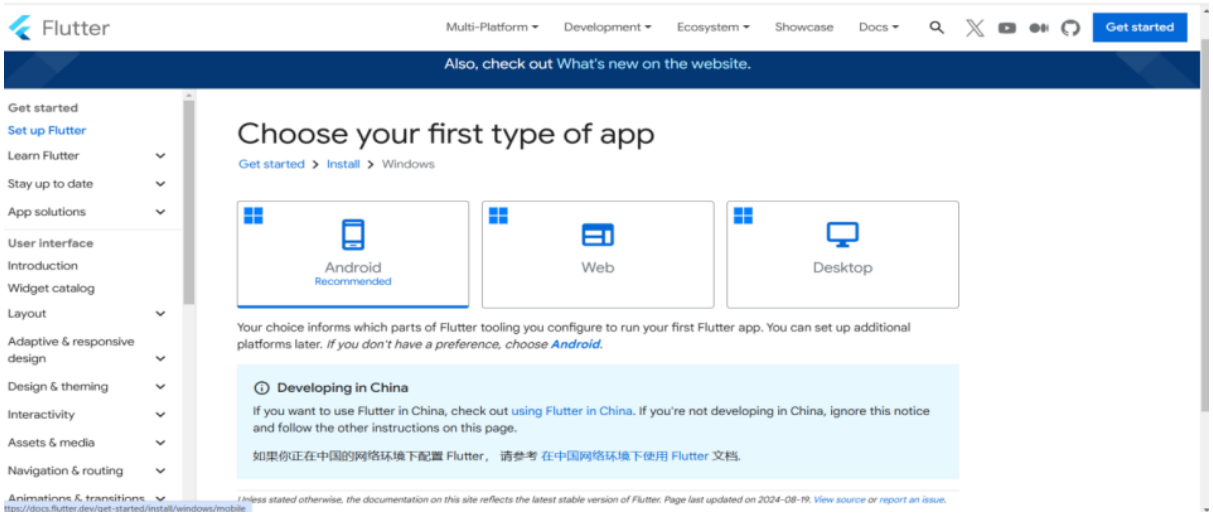
Name:- Vanshika Ambwani Class:- D15A Roll:No: - 02

AIM: - Installation and Configuration of Flutter Environment.

Step 1: Go to the official Flutter website: https://docs.flutter.dev/get-started/install

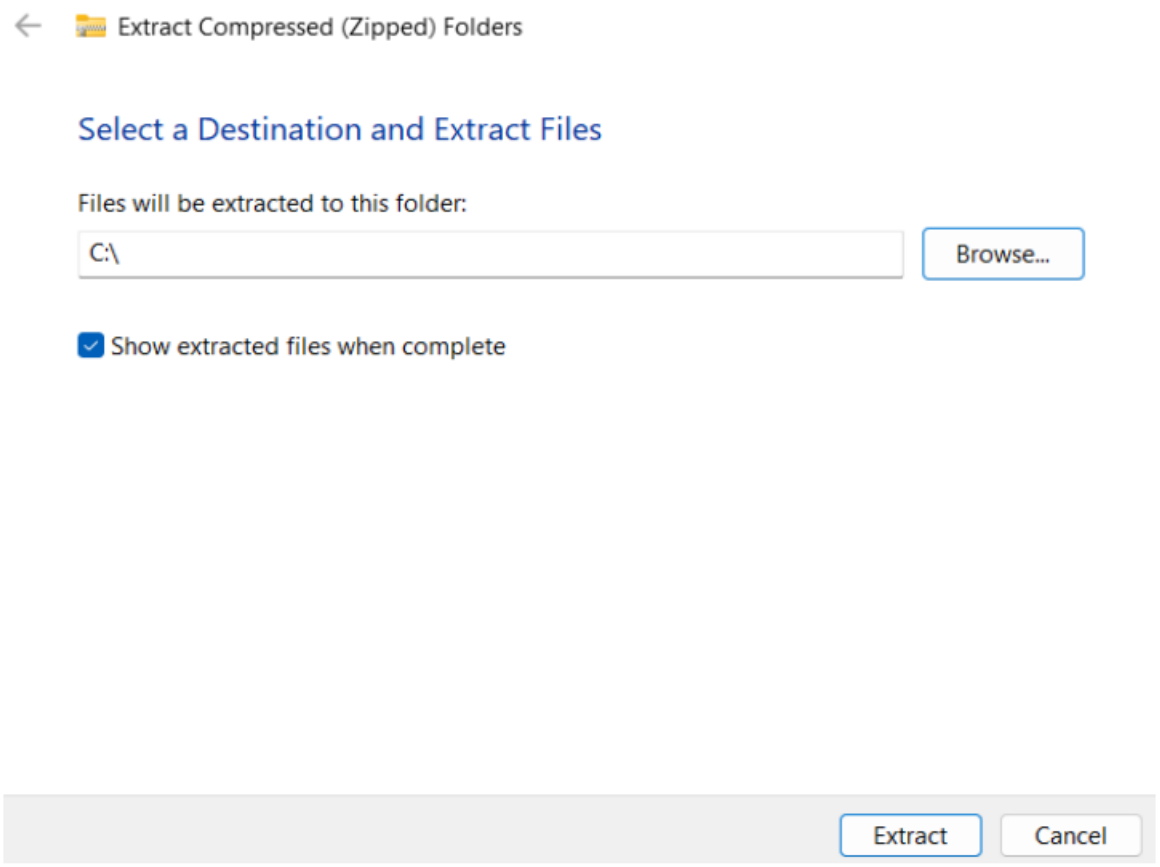


Step 2: To download the latest Flutter SDK, click on the Windows icon > Android

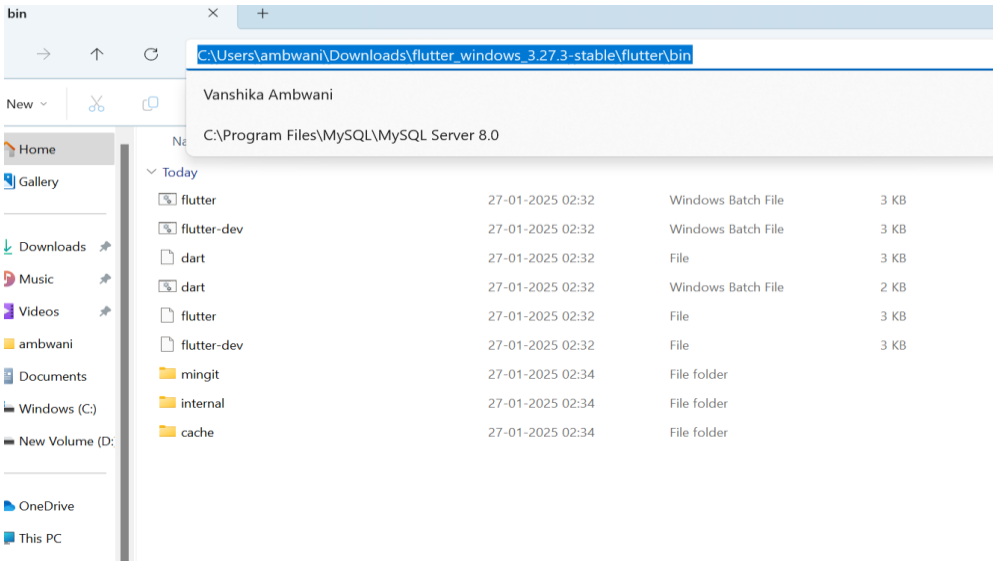


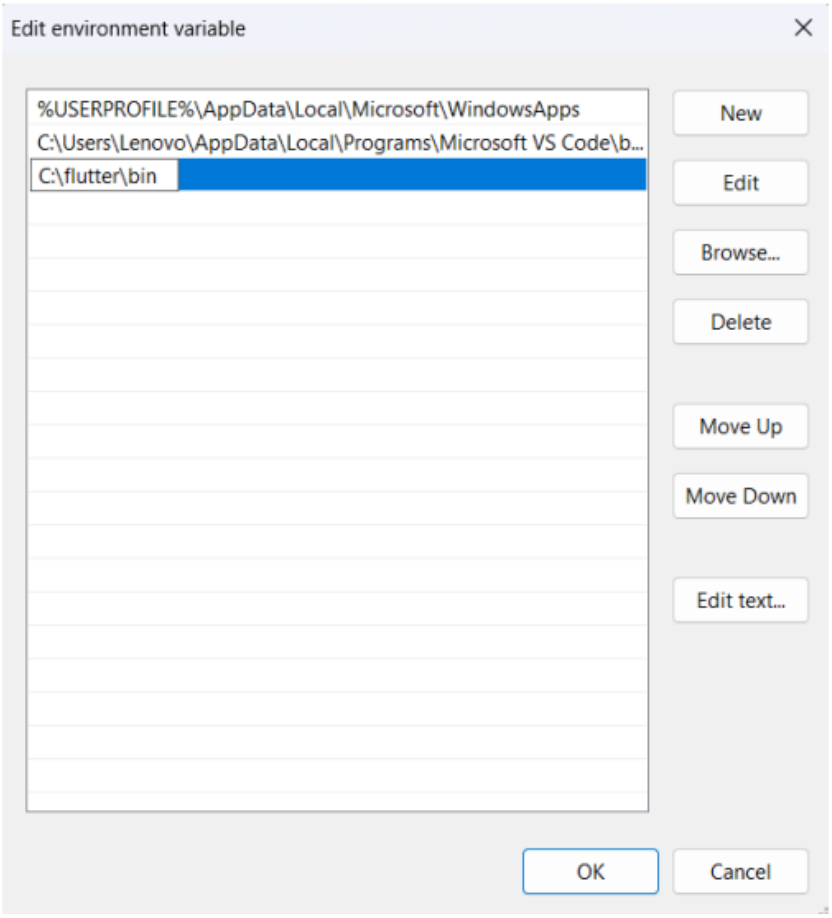
Step 3: For Windows, download the stable release (a .zip file).

Step 4: Extract the ZIP file to a folder (e.g., C:\flutter)



Step 5 :- Add Flutter to System PATH Right-click on the Start Menu > System > Advanced system settings > Environment Variables. Under System Variables, find Path and click Edit. Add the full path to the flutter/bin directory (e.g., C:\flutter\bin).





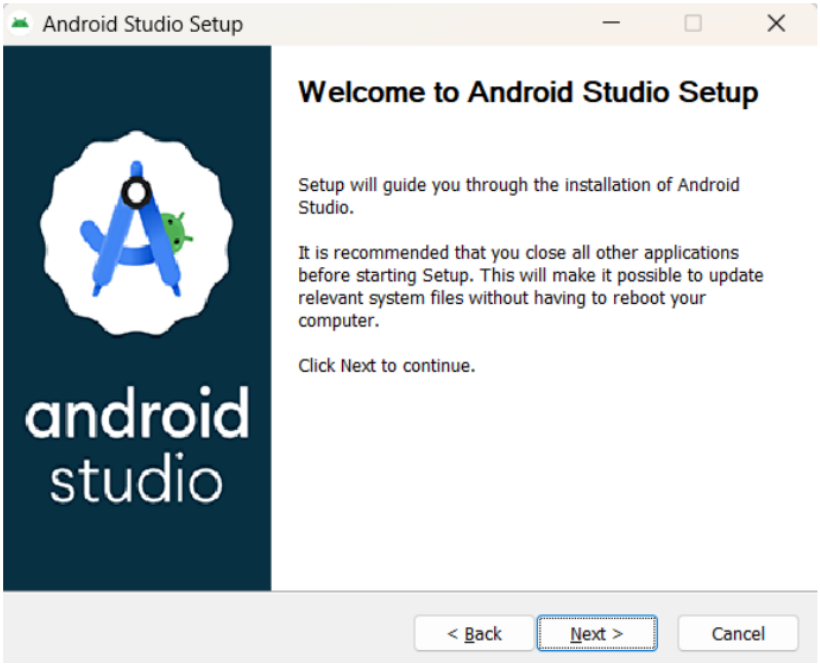
Step 6 : - Now, run the $ flutter command in command prompt

```
  install           Install a Flutter app on an attached device.
  logs              Show log output for running Flutter apps.
  screenshot        Take a screenshot from a connected device.
  symbolize         Symbolize a stack trace from an AOT-compiled Flutter app.

Run "flutter help <command>" for more information about a command.
Run "flutter help -v" for verbose help output, including less commonly used options.

                Welcome to Flutter! - https://flutter.dev

The Flutter tool uses Google Analytics to anonymously report feature usage
statistics and basic crash reports. This data is used to help improve
Flutter tools over time.

Flutter tool analytics are not sent on the very first run. To disable
reporting, type 'flutter config --no-analytics'. To display the current
setting, type 'flutter config'. If you opt out of analytics, an opt-out
event will be sent, and then no further information will be sent by the
Flutter tool.

By downloading the Flutter SDK, you agree to the Google Terms of Service.
The Google Privacy Policy describes how data is handled in this service.

Moreover, Flutter includes the Dart SDK, which may send usage metrics and
crash reports to Google.

Read about data we send with crash reports:
https://flutter.dev/to/crash-reporting

See Google's privacy policy:
```

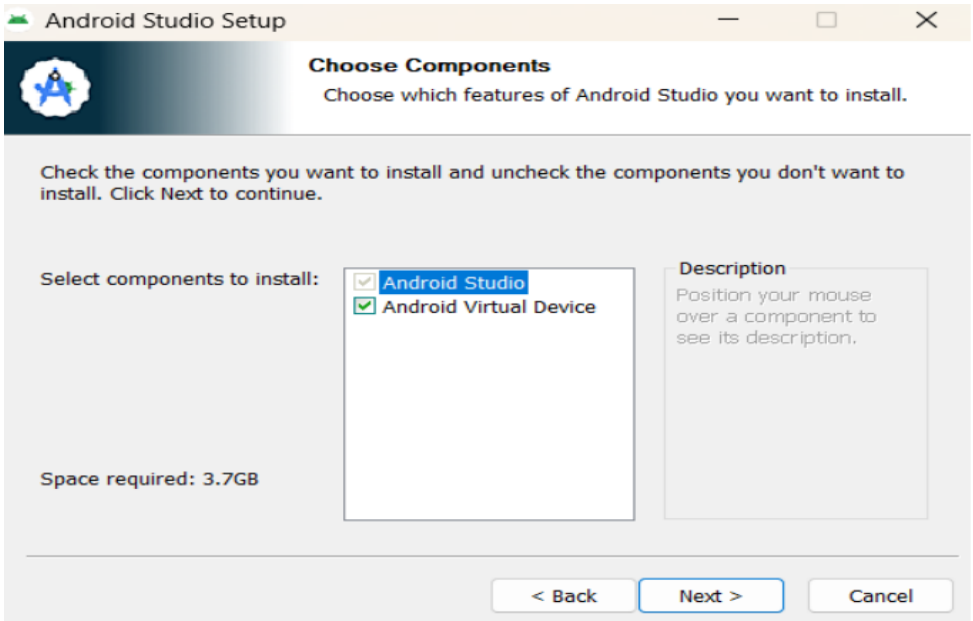
Step 7:- Run the $ flutter doctor command. This command checks for all the requirements of Flutter app development and displays a report of the status of your Flutter installation

Step 8 : - Go to Android Studio and download the installer

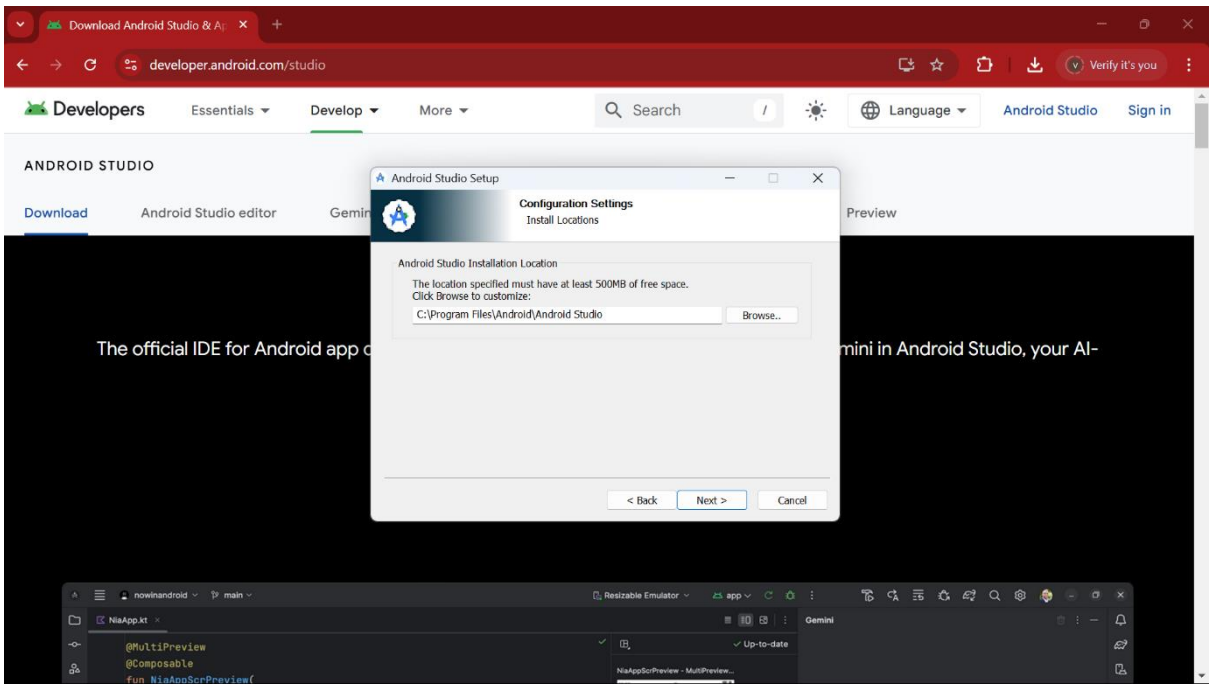
Step 8.1: - When the download is complete, open the .exe file and run it. You will get the following dialog box



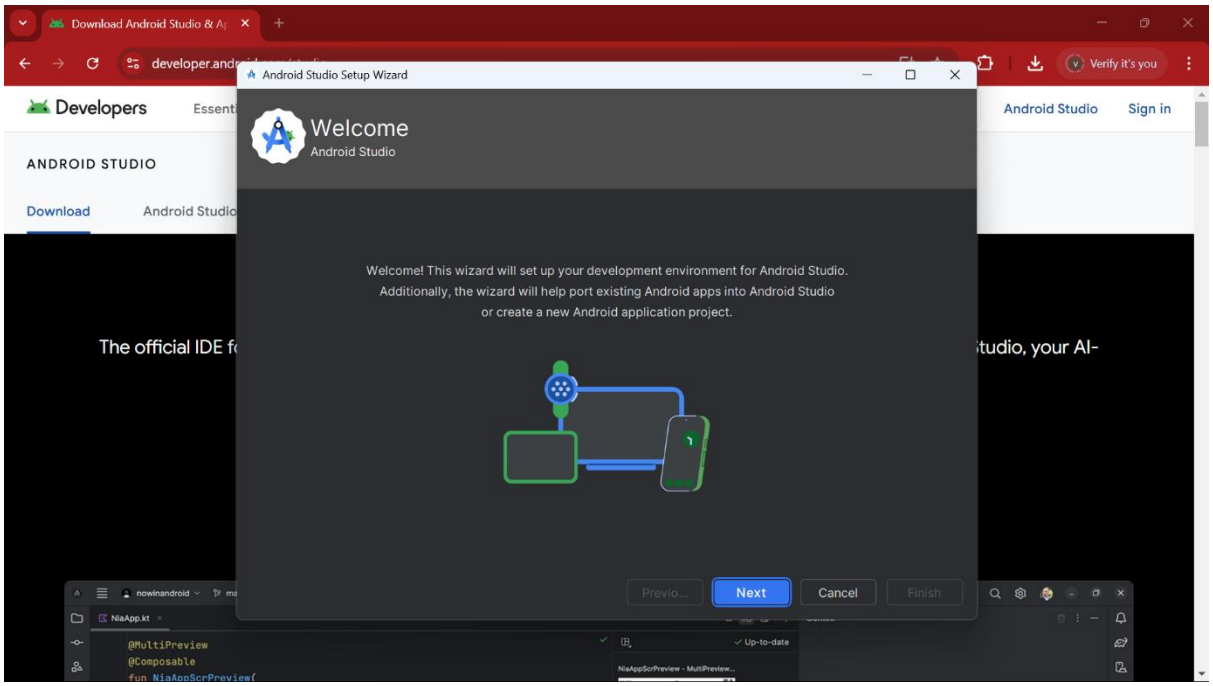
Step 8.2: - Select all the Checkboxes and Click on ‘Next’ Button.

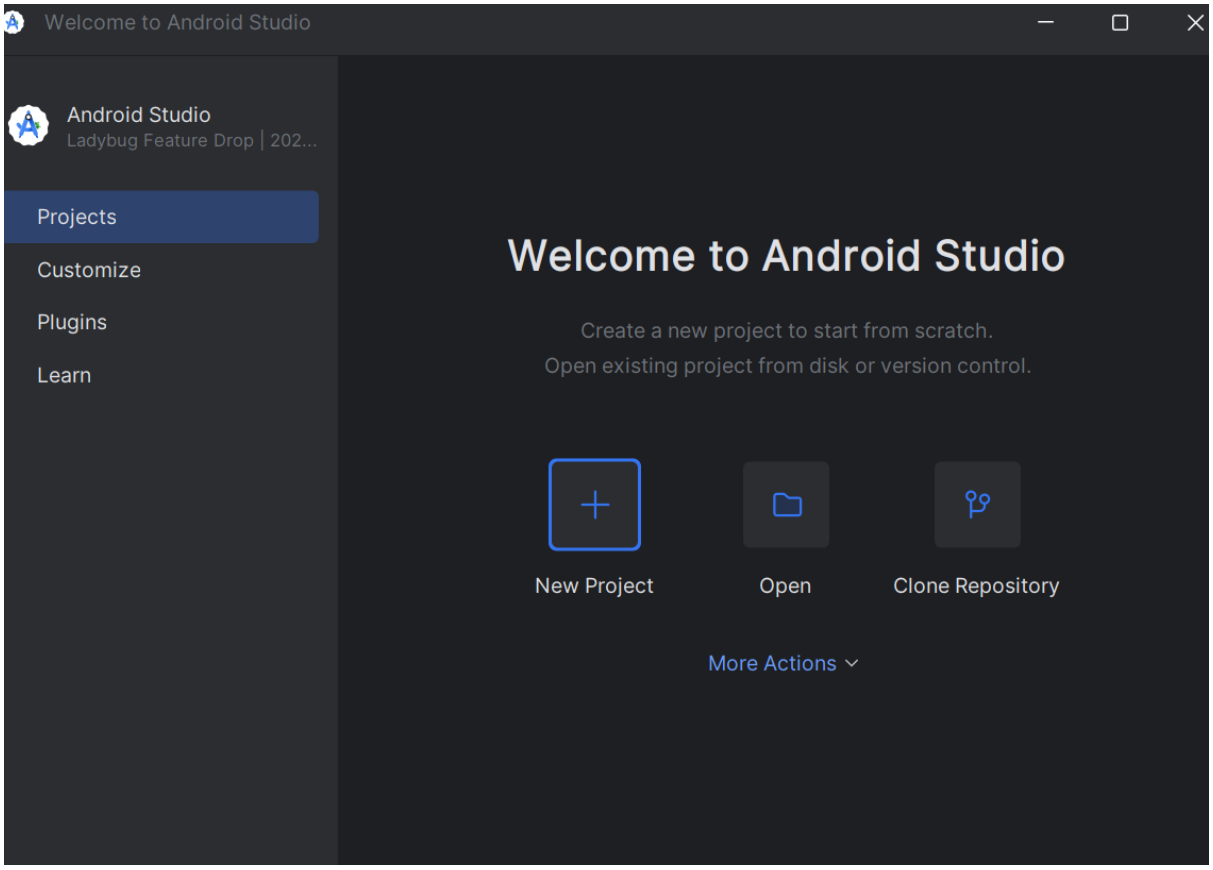


Step 8.3: - Change the destination as per your convenience and click on ‘Next’ Button.



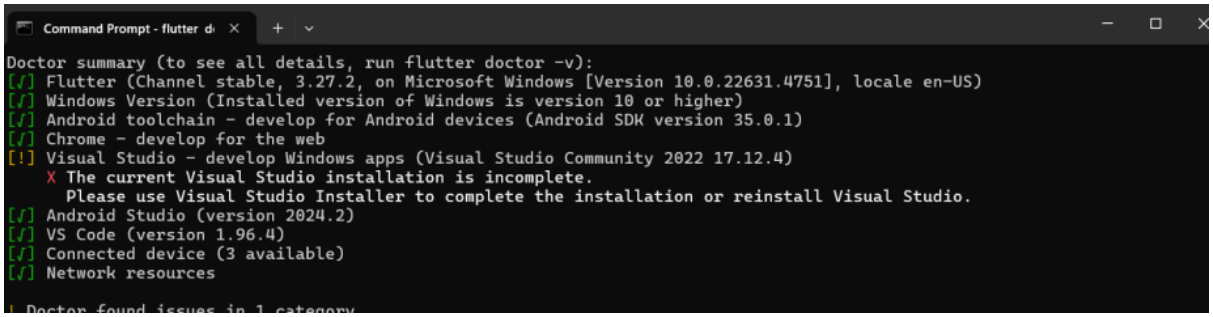
Step 8.4: - Follow the steps of the installation wizard. Once the installation wizard completes, you will get the following screen.

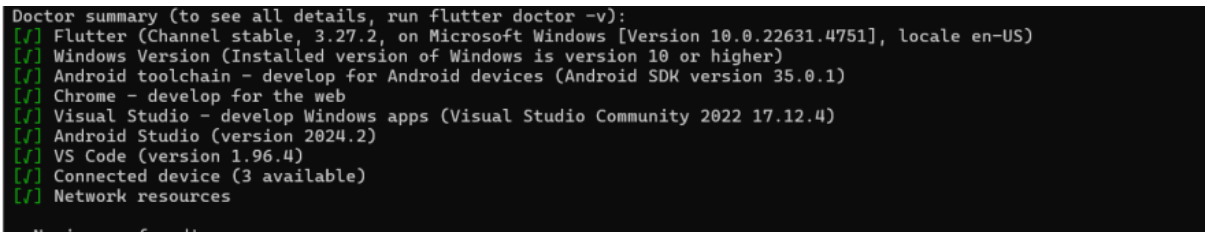




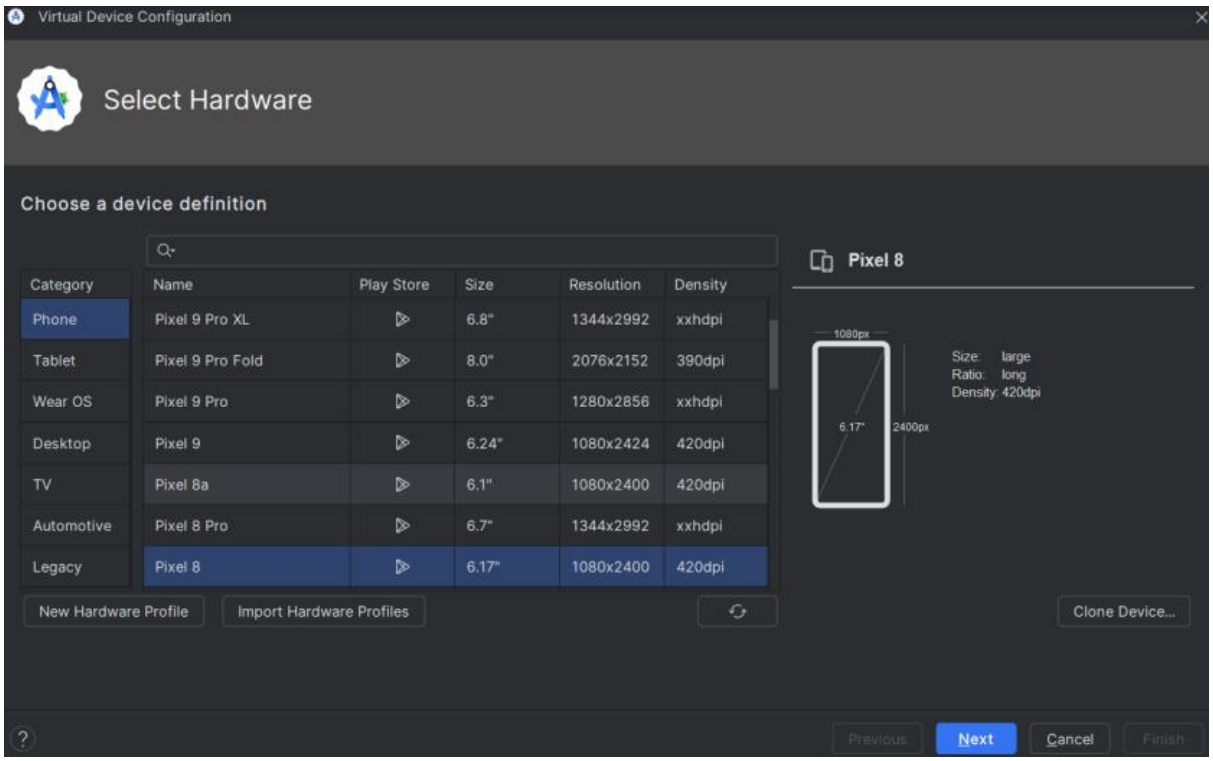
Step 8.5: - Go to Preferences > Appearance & Behavior > System Settings > Android SDK. Select the SDK Tools tab and check Android SDK Command-line Tools and Install it.

Step 9: - Open a terminal and run the following command

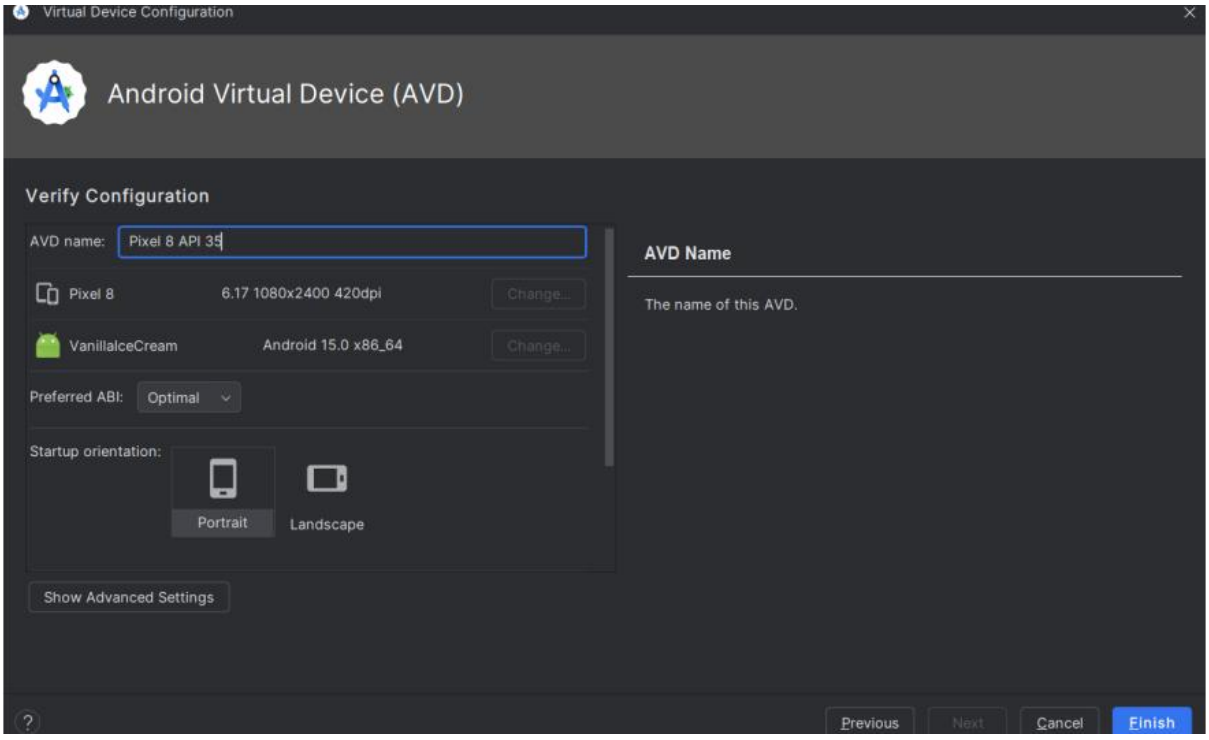




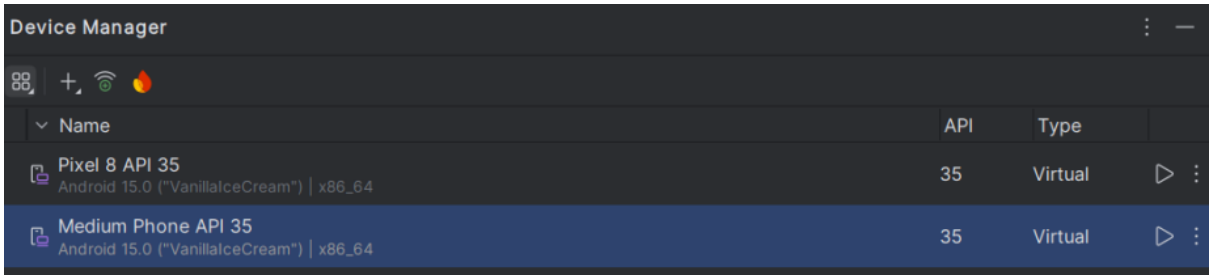
Step 10: - Next, you need to set up an Android emulator. It is responsible for running and testing the Flutter application

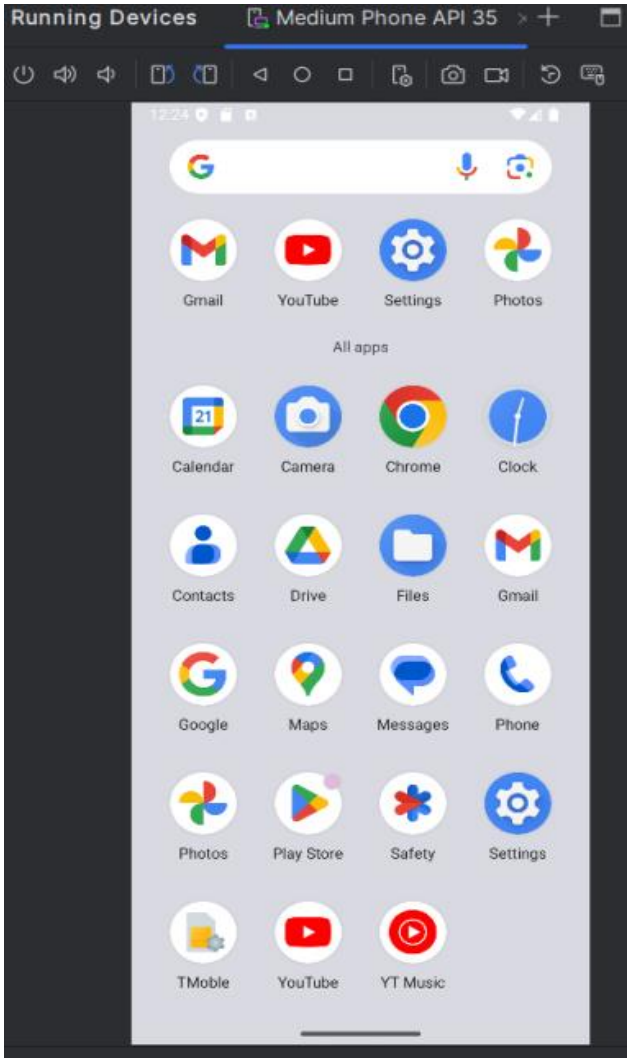


Step 10.1: - Open Android Studio and go to Tools > AVD Manager. Create a new virtual device.



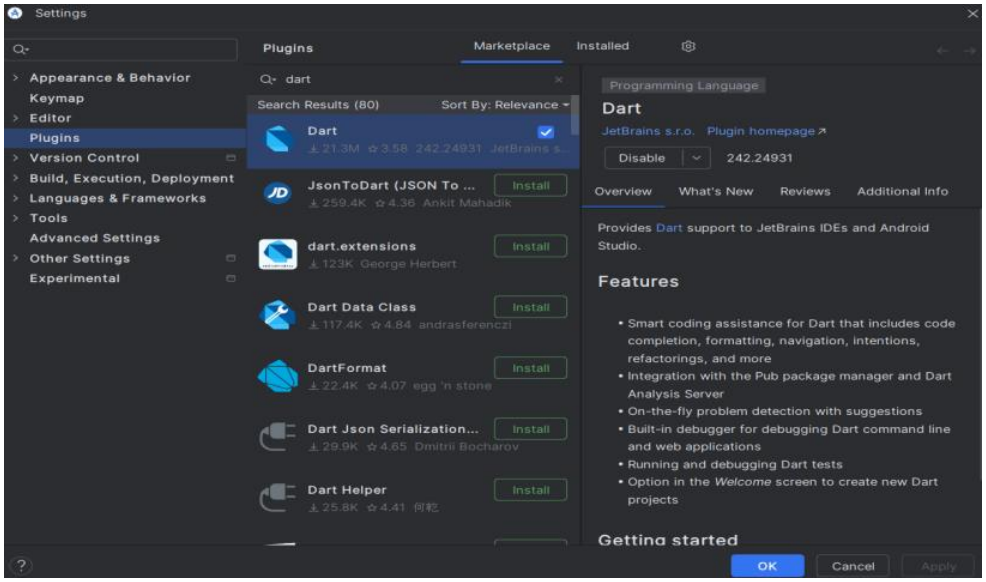
Step 10.2: - Click on the icon pointed into the red color rectangle. The Android emulator displayed as below screen





Step 11: - Now, install Flutter and Dart plugin for building Flutter application in Android Studio. These plugins provide a template to create a Flutter application, give an option to run and debug Flutter application in the Android Studio itself

Step 11.1: - Open the Android Studio and then go to File->Settings->Plugins. Now, search the Flutter plugin. If found, select Flutter plugin and click install



Step 11.2: - Restart the Android Studio

Step 12: - Go to File > New Project > Create Flutter Project, then select the project name and location, and click Next to proceed.

MAD & PWA Lab

Experiment No.	02
Experiment Title.	To design Flutter UI by including common widgets.
Roll No.	01
Name	Vanshika Ambwani
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

Experiment No. 2

Aim:-

To design Flutter UI by including common widgets.

Input Widgets:-

1.TextField: Allows text input.

```
TextField(
decoration: InputDecoration(labelText: 'Enter Name', border:
OutlineInputBorder()),
)
```

2. Button Widgets:

a. ElevatedButton: Raised button with elevation.

ElevatedButton(

onPressed: () {}, child: Text('Click Me'),

)

b. TextButton: Flat button with no elevation.

TextButton(onPressed: () {}, child: Text('Click Me'),

)

c. IconButton: A button with an icon.

IconButton(

icon: Icon(Icons.add), onPressed: () {},

)

D. Navigation & Routing Widgets

1. Navigator: Handles screen transitions.
   Navigator.push(context, MaterialPageRoute(builder: (context) =>
   NewScreen())); 2. BottomNavigationBar: Used for switching between different screens. BottomNavigationBar(
   items: [BottomNavigationBarItem(icon: Icon(Icons.home), label:
   'Home'),
   BottomNavigationBarItem(icon: Icon(Icons.settings), label: 'Settings'),
   ],
)

E. Interactive & Gestures GestureDetector: Detects taps, swipes, and other gestures. GestureDetector(

onTap: () => print('Tapped!'),

child: Container(color: Colors.blue, height: 100, width: 100),

)

Code:-

Profile

```
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';
import 'package:foodie/theme/text_styles.dart';

class ProfileTitleRow extends StatelessWidget {
  const ProfileTitleRow({
    super.key,
    required this.icon,
    required this.label,
  });

  final String icon;
  final String label;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 15),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          SvgPicture.asset(icon),
          const SizedBox(
            width: 15,
          ),
          Expanded(
            child: Text(
```

```
            label,
            style: TextStyles.titleProfile,
          ),
        ),
      ],
    ),
  );
 }
}
```

Homepage

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:foodie/router/router_context_extension.dart';
import 'package:foodie/services/points/points.dart';
import '../../../../../common/is_saved_material_button.dart';
import '../../../domain/recipe.dart';
import 'package:cached_network_image/cached_network_image.dart';
import 'package:shimmer/shimmer.dart';
import '../../../../../theme/theme.dart';

class RecipeWidget extends ConsumerWidget {
  const RecipeWidget({
    super.key,
    required this.recipe,
  });

  final Recipe recipe;

  @override
```

```
Widget build(BuildContext context, WidgetRef ref) {
  return GestureDetector(
    onTap: () => context.pushDetailPage(recipe: recipe),
    child: Stack(
      alignment: Alignment.bottomCenter,
      children: [
        Container(
          height: 120,
          alignment: Alignment.bottomCenter,
          decoration: BoxDecoration(
              borderRadius: BorderRadius.circular(
                16,
              ),
              color: ThemeColors.main),
        ),
        Padding(
          padding: const EdgeInsets.symmetric(horizontal: 15),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            mainAxisAlignment: MainAxisAlignment.spaceAround,
            children: [
              Align(
                alignment: Alignment.topCenter,
                child: Stack(
                  children: [
                    ClipRRect(
                      borderRadius: BorderRadius.circular(16),
                      child: CachedNetworkImage(
                        width: 150,
                        height: 150,
```

```
            fit: BoxFit.cover,
            imageUrl: recipe.imageUrl,
            placeholder: (context, url) => Shimmer.fromColors(
              baseColor: Colors.black26,
              highlightColor: Colors.black12,
              child: Container(
                color: ThemeColors.white,
              ),
            ),
          ),
        ),
        Padding(
          padding: const EdgeInsets.all(10),
          child: IsSavedMaterialButton(
            recipe: recipe,
          ),
        ),
      ],
    ),
  ),
),
Text(
  recipe.name,
  maxLines: 2,
  style: TextStyles.recipeName,
),
Padding(
  padding: const EdgeInsets.only(bottom: 5),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
```

```
                Text(
                  '${Points.recipePoints} points',
                  style: TextStyles.points,
                ),
                Text(
                  '${recipe.time != null ? (recipe.time != 0 ? recipe.time : '') : ''}
${recipe.time != null ? (recipe.time != 0 ? 'mins' : '') : ''}',
                  style: TextStyles.time,
                ),
              ],
            ),
          ),
        ],
      ),
    )
  ],
 ),
 );
 }
}
```

Saved

```
import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:foodie/constants/storage_box_constants.dart';
import 'package:foodie/providers/providers.dart';
import 'package:foodie/router/router_context_extension.dart';
import 'package:shimmer/shimmer.dart';
import '../../../../constants/string_constants.dart';
```

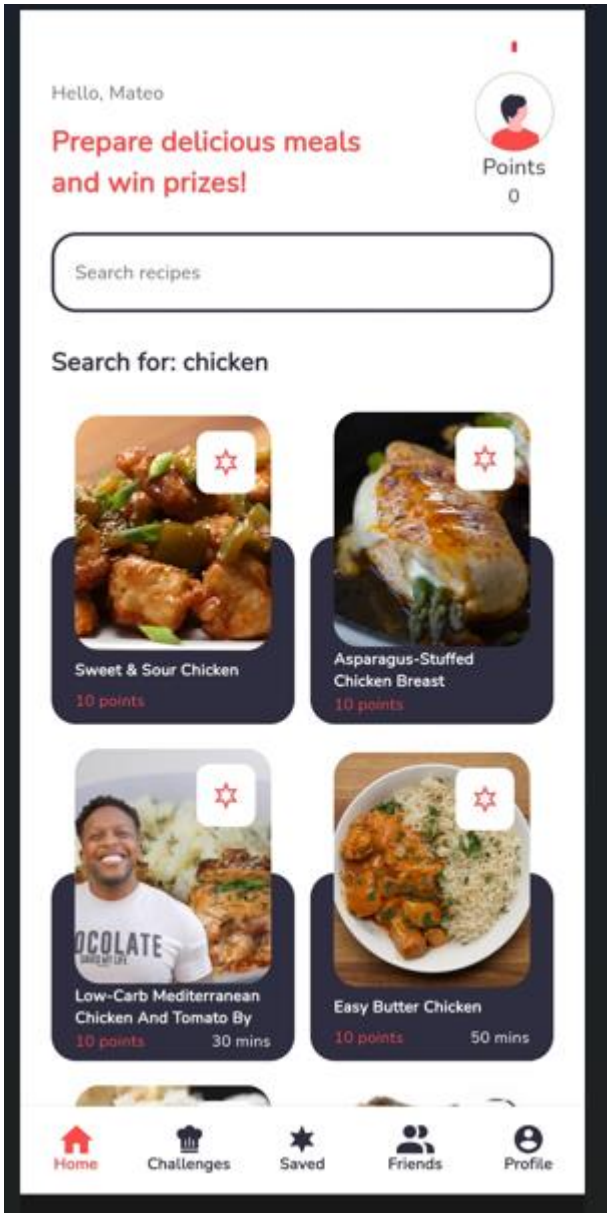
```
import '../../../../services/points/points.dart';
import '../../../../theme/theme.dart';
import '../../domain/recipe.dart';

class SavedRecipesPage extends ConsumerWidget {
  const SavedRecipesPage({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final storageService = ref.watch(storageServiceProvider);
    final imageSize = MediaQuery.of(context).size.width * 0.3;
    final favoriteBoxSize =
        storageService.getLength<Recipe>(StorageBox.favoritesBox);
    return WillPopScope(
      onWillPop: () async => false,
      child: Scaffold(
        appBar: AppBar(
          automaticallyImplyLeading: false,
          title: Text(
            StringConstants.savedRecipes,
            style: TextStyles.title,
          ),
        ),
        body: SafeArea(
          child: favoriteBoxSize == 0
              ? Center(
                  heightFactor: MediaQuery.sizeOf(context).height * 0.5,
                  child: Text(
                    StringConstants.noSavedRecipes,
                    style: TextStyles.subtitle,
```

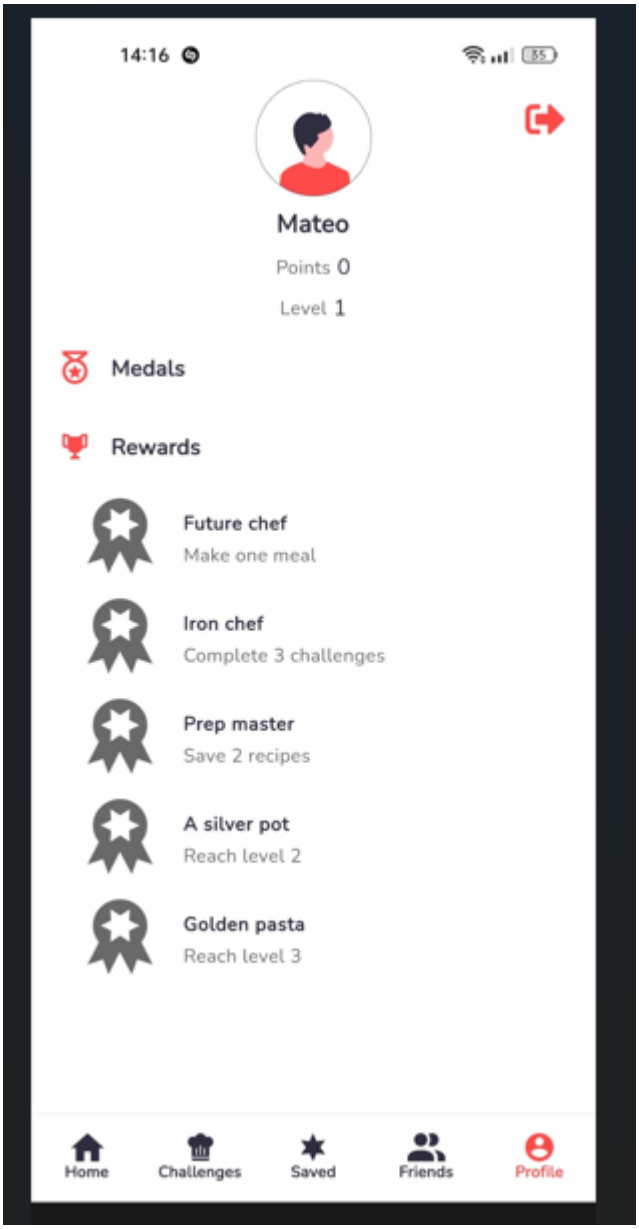
```
      ),
    )
  : Column(
      children: [
        Expanded(
          child: ListView.builder(
              shrinkWrap: true,
              itemCount: favoriteBoxSize,
              itemBuilder: (context, index) {
                final recipe = storageService
                    .getAll<Recipe>(StorageBox.favoritesBox)[index];
                return Padding(
                  padding: const EdgeInsets.fromLTRB(20, 20, 20, 0),
                  child: GestureDetector(
                    onTap: () =>
                        context.pushDetailPage(recipe: recipe),
                    child: Row(
                      children: [
                        ClipRRect(
                          borderRadius: BorderRadius.circular(16),
                          child: CachedNetworkImage(
                            width: imageSize,
                            height: imageSize,
                            fit: BoxFit.cover,
                            imageUrl: recipe.imageUrl,
                            placeholder: (context, url) =>
                                Shimmer.fromColors(
                              baseColor: Colors.black26,
                              highlightColor: Colors.black12,
                              child: Container(
```

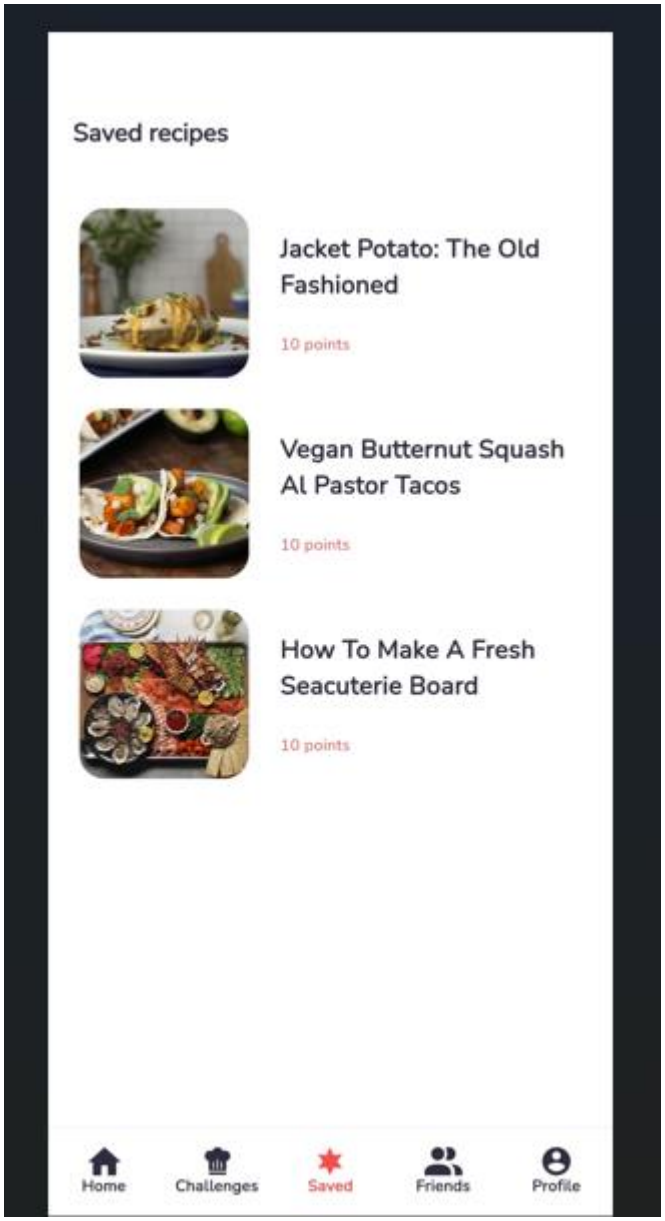
```
                color: ThemeColors.white,
              ),
            ),
          ),
        ),
        const SizedBox(
          width: 20,
        ),
        Expanded(
          child: Column(
            crossAxisAlignment:
                CrossAxisAlignment.start,
            children: [
              Text(
                recipe.name,
                style: TextStyles.title,
              ),
              const SizedBox(
                height: 20,
              ),
              Text('${Points.recipePoints} points',
                  style: TextStyles.points)
            ],
          ),
        )
      ],
    ),
  ),
);
}),
```

```
                )
              ],
            ),
          ),
        ),
      );
    }
}
```

Output:-


Hello, Mateo
Prepare delicious meals and win prizes!
Points
0
Search recipes
Search for: chicken
Sweet & Sour Chicken
10 points
Asparagus-Stuffed Chicken Breast
10 points
Low-Carb Mediterranean Chicken And Tomato By
10 points
30 mins
Easy Butter Chicken
10 points
50 mins
Home
Challenges
Saved
Friends
Profile


14:16
Mateo
Points 0
Level 1
Medals
Rewards
Future chef
Make one meal
Iron chef
Complete 3 challenges
Prep master
Save 2 recipes
A silver pot
Reach level 2
Golden pasta
Reach level 3
Home
Challenges
Saved
Friends
Profile


Saved recipes
Jacket Potato: The Old Fashioned
10 points
Vegan Butternut Squash Al Pastor Tacos
10 points
How To Make A Fresh Seacuterie Board
10 points
Home
Challenges
Saved
Friends
Profile

Experiment No.	03
Experiment Title.	To include icons, images, fonts in Flutter app
Roll No.	02
Name	Vanshika Ambwani
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

Experiment No.3

Aim:-

To include images, fonts in flutter app

Theory:-

Images are an essential part of UI design, and Flutter supports adding both local and network images.

A) Local images can be stored in the project directory and loaded into the app.

Steps to add Local images

- Create an assets folder in the root directory.
- Store images inside the assets folder. Declare assets in pubspec.yaml under the flutter section:
  flutter:
  assets:
  - assets/image1.png
  - assets/images/image2.jpg

B) Network Images Flutter allows displaying images from the internet using Image.network():

Image.network('https://example.com/image.jpg')

Font Awesome provides a vast collection of scalable vector icons that behave like fonts. These icons can be used in Flutter via the font_awesome_flutter package, which integrates Font Awesome's font-based icons seamlessly into the app.

Widget properties:-

1)image

- width: Sets image width.
- height: Sets image height.
- fit: Controls how image fits (e.g., BoxFit.cover, BoxFit.fill).
- alignment: Aligns the image inside the container.
- color: Applies a color filter.
- opacity: Controls image transparency.
- loadingBuilder: Handles loading states.
- errorBuilder: Handles image load errors.

2)font

- size: Adjusts icon size.
- color: Sets icon color.

semanticLabel: Adds an accessibility label for screen readers.

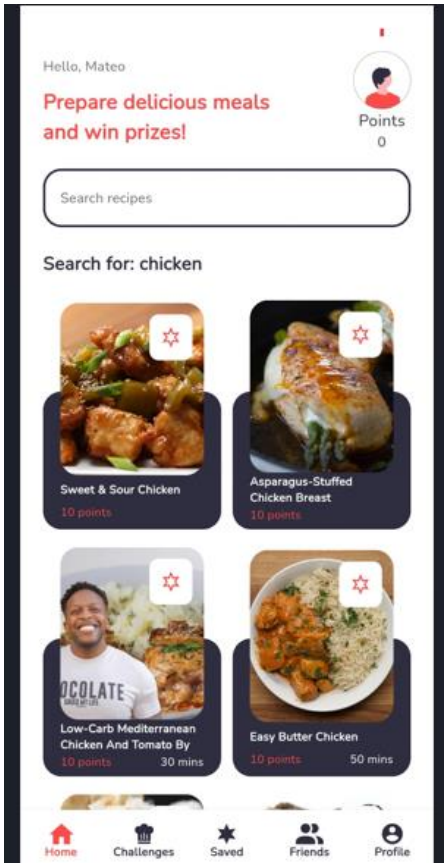
Code:-

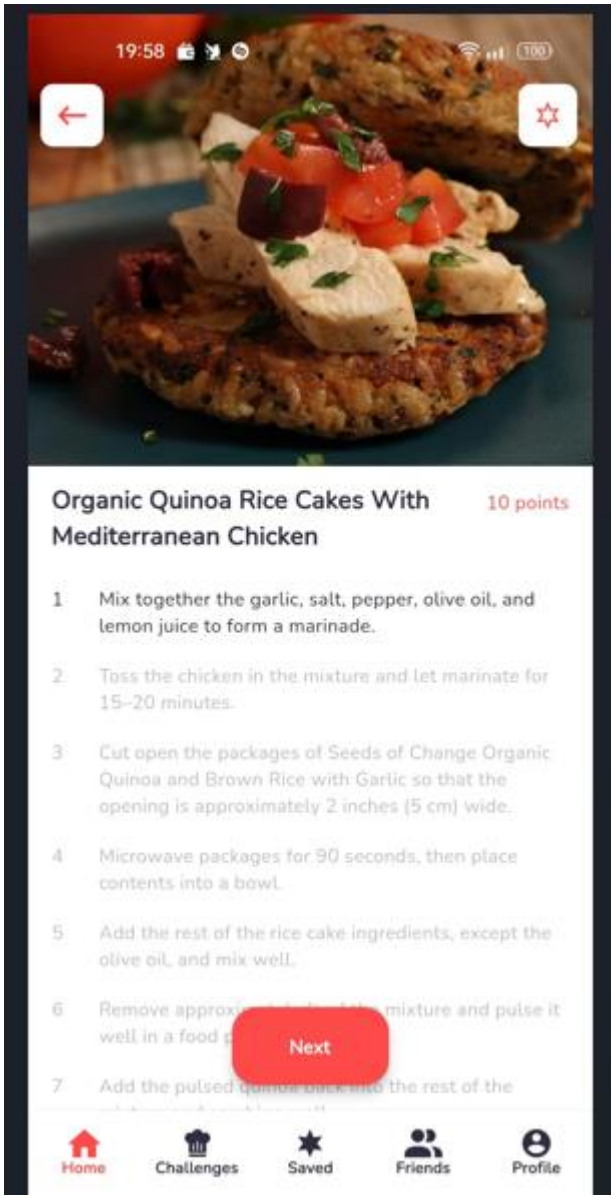
```
// ignore_for_file: library_private_types_in_public_api

class Assets {
  static _Avatars avatars = _Avatars();
  static _Challenges challenges = _Challenges();
  static _Icons icons = _Icons();
  static _Images images = _Images();
}

class _Avatars {
  String avatar0 = 'assets/avatars/avatar0.svg';
  String avatar1 = 'assets/avatars/avatar1.svg';
  String avatar2 = 'assets/avatars/avatar2.svg';
  String avatar3 = 'assets/avatars/avatar3.svg';
  String avatar4 = 'assets/avatars/avatar4.svg';
  String avatar5 = 'assets/avatars/avatar5.svg';
  String avatar6 = 'assets/avatars/avatar6.svg';
  String avatar7 = 'assets/avatars/avatar7.svg';
}

class _Challenges {
  String challenge1 = 'assets/challenges/component1.svg';
  String challenge2 = 'assets/challenges/component2.svg';
  String challenge3 = 'assets/challenges/component3.svg';
  String challenge4 = 'assets/challenges/component4.svg';
}

class _Icons {
```

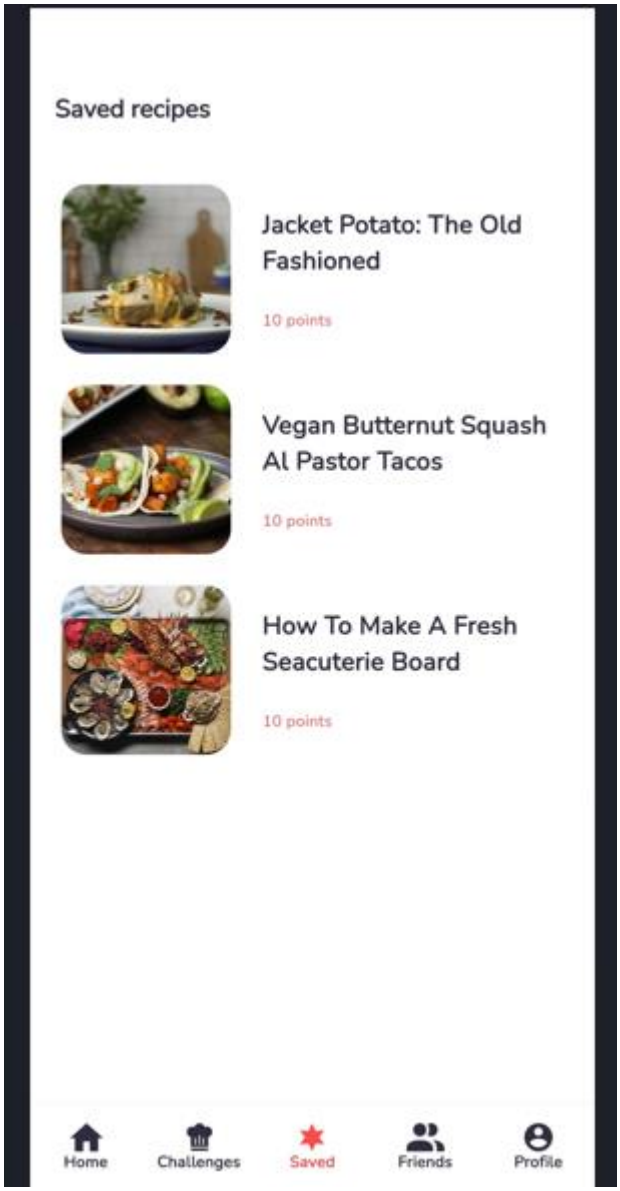
```
  String add = 'assets/icons/add.svg';
  String addFriend = 'assets/icons/addFriend.svg';
  String back = 'assets/icons/back.svg';
  String challenges = 'assets/icons/challenges.svg';
  String check = 'assets/icons/check.svg';
  String friends = 'assets/icons/friends.svg';
  String home = 'assets/icons/home.svg';
  String image = 'assets/icons/image.svg';
  String profile = 'assets/icons/profile.svg';
  String paperPlane = 'assets/icons/paperPlane.svg';
  String replace = 'assets/icons/replace.svg';
  String save = 'assets/icons/save.svg';
  String saved = 'assets/icons/saved.svg';
  String search = 'assets/icons/search.svg';
  String share = 'assets/icons/share.svg';
  String subtract = 'assets/icons/subtract.svg';
  String trophy = 'assets/icons/trophy.svg';
  String time = 'assets/icons/time.svg';
  String medal = 'assets/icons/medal.svg';
  String unreceivedBadge = 'assets/icons/unreceivedBadge.svg';
  String receivedBadge = 'assets/icons/receivedBadge.svg';
  String signOut = 'assets/icons/signOut.svg';
  String randomDices = 'assets/icons/randomDices.svg';
  String fingerPointing = 'assets/icons/fingerPointing.svg';
}

class _Images {
  String logo = 'assets/onboarding/logo.svg';
  String onboarding0 = 'assets/onboarding/onboarding0.svg';
  String onboarding1 = 'assets/onboarding/onboarding1.svg';
```

```
  String onboarding2 = 'assets/onboarding/onboarding2.svg';
  String onboarding3 = 'assets/onboarding/onboarding3.svg';
}
```

Ouput:-




19:58
Organic Quinoa Rice Cakes With Mediterranean Chicken
10 points
1 Mix together the garlic, salt, pepper, olive oil, and lemon juice to form a marinade.
2 Toss the chicken in the mixture and let marinate for 15–20 minutes.
3 Cut open the packages of Seeds of Change Organic Quinoa and Brown Rice with Garlic so that the opening is approximately 2 inches (5 cm) wide.
4 Microwave packages for 90 seconds, then place contents into a bowl.
5 Add the rest of the rice cake ingredients, except the olive oil, and mix well.
Next
Home
Challenges
Saved
Friends
Profile


Saved recipes
Jacket Potato: The Old Fashioned
10 points
Vegan Butternut Squash Al Pastor Tacos
10 points
How To Make A Fresh Seacuterie Board
10 points
Home
Challenges
Saved
Friends
Profile

MAD & PWA Lab

Journal

Experiment No.	04
Experiment Title.	To create an interactive Form using form widget
Roll No.	02
Name	Vanshika Ambwani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

Experiment 4

Aim: Guidelines To create an interactive Form using form widget

Code:

login page

```
import 'package:cloud_firestore/cloud_firestore.dart';

import 'package:firebase_auth/firebase_auth.dart';

import 'package:flutter/cupertino.dart';

import 'package:flutter/material.dart';

// import 'package:';

// import 'sign_up_page.dart';

// import
'package:flutter_firebase/features/user_auth/presentation/pages/sign_up_page.dart';

// import
'package:flutter_firebase/features/user_auth/presentation/widgets/form_container_wi
dget.dart';

// import 'package';

// import 'package:flutter_firebase/global/common/toast.dart';

import 'package:font_awesome_flutter/font_awesome_flutter.dart';

import
'package:login_auth_new/features/user_auth/firebase_auth_implementation/firebase
_auth_services.dart';

import
'package:login_auth_new/features/user_auth/presentation/pages/login_page.dart';

import
'package:login_auth_new/features/user_auth/presentation/widgets/form_container_w
idget.dart';

import 'package:login_auth_new/global/common/toast.dart';


// import 'package:google_sign_in/google_sign_in.dart';
```

```dart
import
'package:login_auth_new/features/user_auth/presentation/pages/sign_up_page.dart'
;
import 'package:login_auth_new/global/common/toast.dart';


import '../../firebase_auth_implementation/firebase_auth_services.dart';


class LoginPage extends StatefulWidget {
  const LoginPage({super.key});


  @override
  State<LoginPage> createState() => _LoginPageState();
}


class _LoginPageState extends State<LoginPage> {
  bool _isSigning = false;
  final FirebaseAuthService _auth = FirebaseAuthService();
  final FirebaseAuth _firebaseAuth = FirebaseAuth.instance;
  TextEditingController _emailController = TextEditingController();
  TextEditingController _passwordController = TextEditingController();


  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }


  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

```
appBar: AppBar(
  automaticallyImplyLeading: false,
  title: Text("Login"),
),
body: Center(
  child: Padding(
    padding: const EdgeInsets.symmetric(horizontal: 15),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text(
          "Login",
          style: TextStyle(fontSize: 27, fontWeight: FontWeight.bold),
        ),
        SizedBox(
          height: 30,
        ),
        FormContainerWidget(
          controller: _emailController,
          hintText: "Email",
          isPasswordField: false,
        ),
        SizedBox(
          height: 10,
        ),
        FormContainerWidget(
          controller: _passwordController,
          hintText: "Password",
          isPasswordField: true,
        ),
```

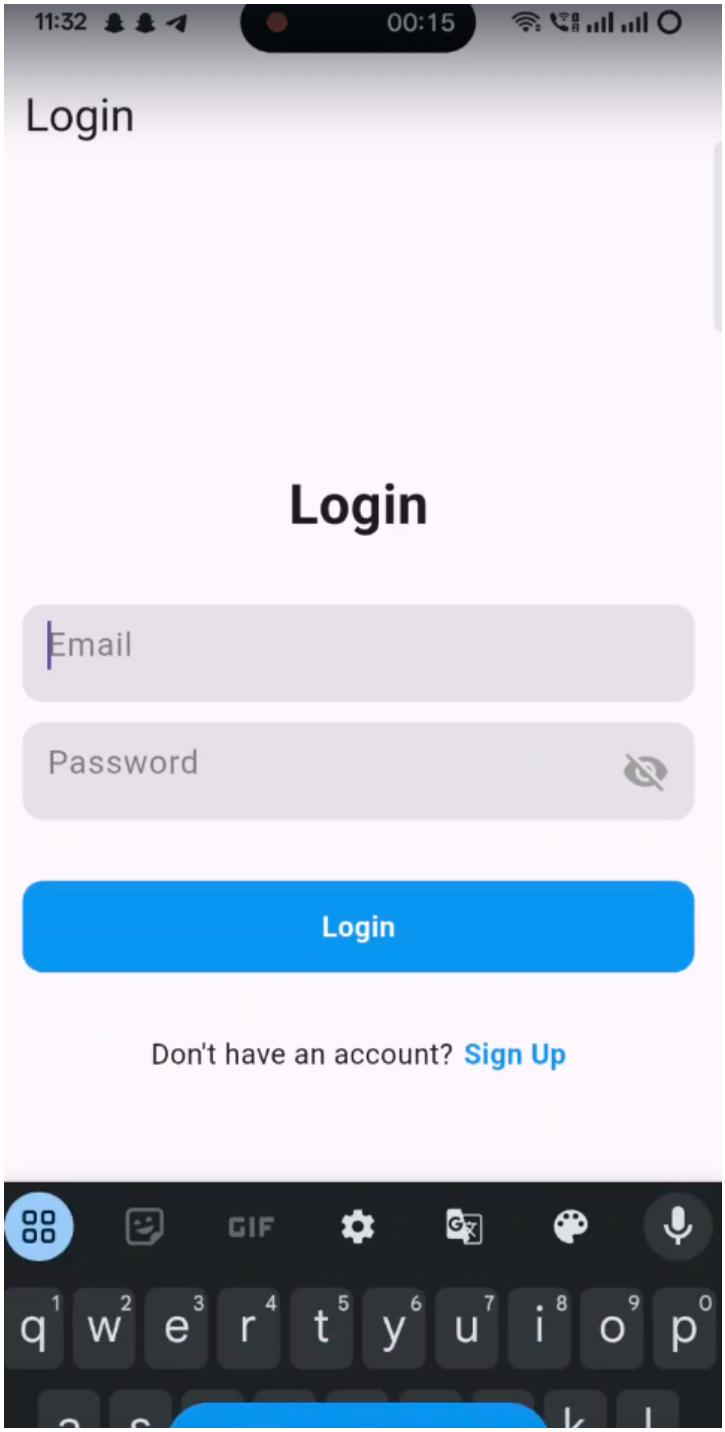
```
SizedBox(
  height: 30,
),
GestureDetector(
  onTap: () {
    _signIn();
  },
  child: Container(
    width: double.infinity,
    height: 45,
    decoration: BoxDecoration(
      color: Colors.blue,
      borderRadius: BorderRadius.circular(10),
    ),
    child: Center(
      child: _isSigning ? CircularProgressIndicator(
        color: Colors.white,) : Text(
        "Login",
        style: TextStyle(
          color: Colors.white,
          fontWeight: FontWeight.bold,
        ),
      ),
    ),
  ),
),
SizedBox(height: 10,),
// GestureDetector(
//   onTap: () {
//     _signInWithGoogle();
```

```
//   },
//   child: Container(
//     width: double.infinity,
//     height: 45,
//     decoration: BoxDecoration(
//       color: Colors.red,
//       borderRadius: BorderRadius.circular(10),
//     ),
//     child: Center(
//       child: Row(
//         mainAxisAlignment: MainAxisAlignment.center,
//         children: [
//           Icon(FontAwesomeIcons.google, color: Colors.white,),
//           SizedBox(width: 5,),
//           Text(
//             "Sign in with Google",
//             style: TextStyle(
//               color: Colors.white,
//               fontWeight: FontWeight.bold,
//             ),
//           ),
//         ],
//       ),
//     ),
//   ),
// ),


SizedBox(
```

```
      height: 20,
    ),

    Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text("Don't have an account?"),
        SizedBox(
          width: 5,
        ),
        GestureDetector(
          onTap: () {
            Navigator.pushAndRemoveUntil(
              context,
              MaterialPageRoute(builder: (context) => SignUpPage()),
                  (route) => false,
            );
          },
          child: Text(
            "Sign Up",
            style: TextStyle(
              color: Colors.blue,
              fontWeight: FontWeight.bold,
            ),
          ),
        ),
      ],
    ),
  ],
),
```

```
        ),
      ),
    );
  }

  void _signIn() async {
    setState(() {
      _isSigning = true;
    });

    String email = _emailController.text;
    String password = _passwordController.text;

    User? user = await _auth.signInWithEmailAndPassword(email, password);

    setState(() {
      _isSigning = false;
    });

    if (user != null) {
      showToast(message: "User is successfully signed in");
      Navigator.pushNamed(context, "/home");
    } else {
      showToast(message: "some error occured");
    }
  }


//   _signInWithGoogle()async{
```

```
//     final GoogleSignIn _googleSignIn = GoogleSignIn();

//     try {

//       final GoogleSignInAccount? googleSignInAccount = await
_googleSignIn.signIn();

//       if(googleSignInAccount != null ){
//         final GoogleSignInAuthentication googleSignInAuthentication = await
//         googleSignInAccount.authentication;

//         final AuthCredential credential = GoogleAuthProvider.credential(
//           idToken: googleSignInAuthentication.idToken,
//           accessToken: googleSignInAuthentication.accessToken,
//         );

//         await _firebaseAuth.signInWithCredential(credential);
//         Navigator.pushNamed(context, "/home");
//       }

//     }catch(e) {
// showToast(message: "some error occured $e");
//     }



//   }



}
```



signup page

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
```

```
// import
'package:flutter_firebase/features/user_auth/firebase_auth_implementation/firebase_
auth_services.dart';

// import 'package:flutter_firebase';

// import
'package:flutter_firebase/features/user_auth/presentation/pages/login_page.dart';

// import
'package:flutter_firebase/features/user_auth/presentation/widgets/form_container_wi
dget.dart';

// import 'package:flutter_firebase/global/common/toast.dart';


import
'package:login_auth_new/features/user_auth/firebase_auth_implementation/firebase
_auth_services.dart';

import
'package:login_auth_new/features/user_auth/presentation/pages/login_page.dart';

import
'package:login_auth_new/features/user_auth/presentation/widgets/form_container_w
idget.dart';

import 'package:login_auth_new/global/common/toast.dart';

import
'package:login_auth_new/features/user_auth/firebase_auth_implementation/firebase
_auth_services.dart';

import
'package:login_auth_new/features/user_auth/presentation/pages/login_page.dart';

import 'package:login_auth_new/global/common/toast.dart';


class SignUpPage extends StatefulWidget {
  const SignUpPage({super.key});


  @override
  State<SignUpPage> createState() => _SignUpPageState();
}


class _SignUpPageState extends State<SignUpPage> {
```

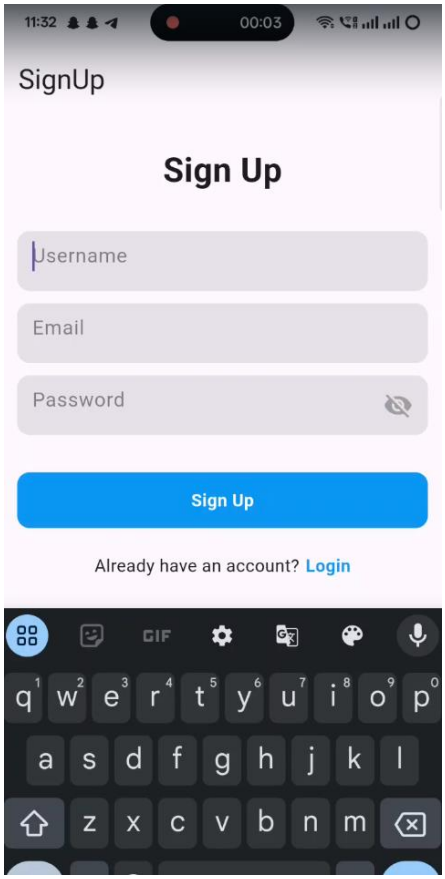
```
final FirebaseAuthService _auth = FirebaseAuthService();

TextEditingController _usernameController = TextEditingController();
TextEditingController _emailController = TextEditingController();
TextEditingController _passwordController = TextEditingController();

bool isSigningUp = false;

@override
void dispose() {
  _usernameController.dispose();
  _emailController.dispose();
  _passwordController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      automaticallyImplyLeading: false,
      title: Text("SignUp"),
    ),
    body: Center(
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 15),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
```

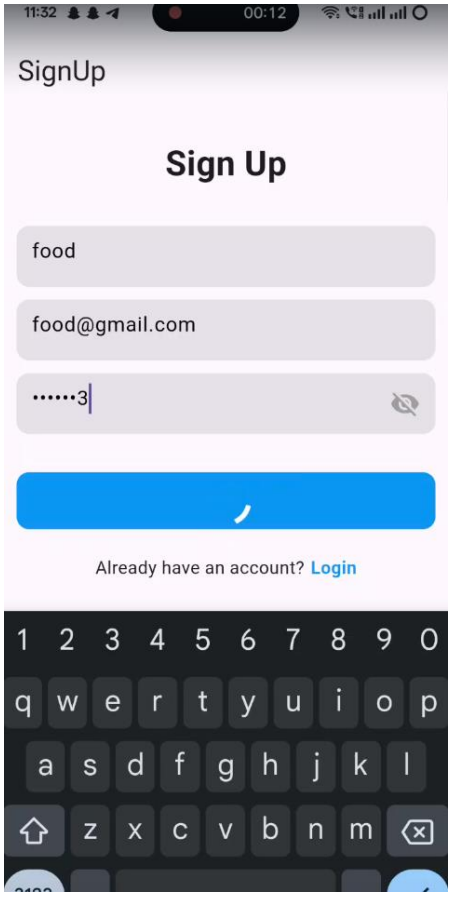
```
  "Sign Up",
  style: TextStyle(fontSize: 27, fontWeight: FontWeight.bold),
),
SizedBox(
  height: 30,
),
FormContainerWidget(
  controller: _usernameController,
  hintText: "Username",
  isPasswordField: false,
),
SizedBox(
  height: 10,
),
FormContainerWidget(
  controller: _emailController,
  hintText: "Email",
  isPasswordField: false,
),
SizedBox(
  height: 10,
),
FormContainerWidget(
  controller: _passwordController,
  hintText: "Password",
  isPasswordField: true,
),
SizedBox(
  height: 30,
),
```

```
            GestureDetector(
              onTap:  (){
                _signUp();


              },
              child: Container(
                width: double.infinity,
                height: 45,
                decoration: BoxDecoration(
                  color: Colors.blue,
                  borderRadius: BorderRadius.circular(10),
                ),
                child: Center(
                    child: isSigningUp ? CircularProgressIndicator(color:
Colors.white,):Text(
                  "Sign Up",
                  style: TextStyle(
                      color: Colors.white, fontWeight: FontWeight.bold),
                )),
              ),
            ),
            SizedBox(
              height: 20,
            ),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Text("Already have an account?"),
                SizedBox(
                  width: 5,
                ),
```

```
                GestureDetector(
                    onTap: () {
                      Navigator.pushAndRemoveUntil(
                          context,
                          MaterialPageRoute(
                              builder: (context) => LoginPage()),
                          (route) => false);
                    },
                    child: Text(
                      "Login",
                      style: TextStyle(
                          color: Colors.blue, fontWeight: FontWeight.bold),
                    ))
              ],
            )
          ],
        ),
      ),
    ),
  );
 }

  void _signUp() async {

setState(() {
  isSigningUp = true;
});

    String username = _usernameController.text;
    String email = _emailController.text;
```

```
    String password = _passwordController.text;

    User? user = await _auth.signUpWithEmailAndPassword(email, password);

setState(() {
  isSigningUp = false;
});
    if (user != null) {
      showToast(message: "User is successfully created");
      Navigator.pushNamed(context, "/home");
    } else {
      showToast(message: "Some error happend");
    }
  }
}
```




11:32
00:12
SignUp
Sign Up
food
food@gmail.com
••••••3
Already have an account? Login
1 2 3 4 5 6 7 8 9 0
q w e r t y u i o p
a s d f g h j k l
z x c v b n m

MAD & PWA Lab

Journal

Experiment No.	05
Experiment Title.	To apply navigation, routing and gestures in Flutter App
Roll No.	02
Name	Vanshika Ambwani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

Experiment No. 5

Aim:-

To apply navigation, routing and gestures in Flutter App

Theory:-

Navigation in Flutter allows users to move between different screens (or pages) in the app. Flutter uses the Navigator widget to handle navigation between routes (screens).

Types of Navigation

● Push Navigation (Forward Navigation) → Moves to a new screen.

● Pop Navigation (Backward Navigation) → Moves back to the previous screen.

● PushReplacement → Replaces the current screen with a new one.

● PushAndRemoveUntil → Moves to a new screen and removes previous screens from the stack.

Types of Routing

1.Direct Route Navigation (MaterialPageRoute)-Used for simple page-topage navigation. 2. Named Routes (Predefined Routes in main.dart)-Defined in the MaterialApp widget and used throughout the app.

Flutter uses the GestureDetector widget to detect user interactions like taps, swipes, pinches, and long presses. This is essential for making an app interactive.

Common Gestures & Their Uses:

● Tap → Detects simple taps on a widget.

● Double Tap → Recognizes double-clicking.

● Long Press → Triggers an action when the user presses and holds.

● Swipe (Drag) → Detects horizontal or vertical dragging.

● Pinch (Zoom In/Out) → Detects two-finger pinch for zooming.

Code:-

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:flutter_svg/flutter_svg.dart';
import 'package:foodie/constants/app_constants.dart';
import 'package:foodie/router/scaffold_with_bottom_nav_controller.dart';
import 'package:go_router/go_router.dart';

import '../constants/string_constants.dart';
import '../features/recipes/presentation/recipe_controller.dart';
import '../providers/providers.dart';
import '../theme/theme.dart';
import 'app_route.dart';

class ScaffoldWithBottomNavBar extends ConsumerStatefulWidget {
  const ScaffoldWithBottomNavBar({
    required this.child,
    super.key,
  });
  final Widget child;

  @override
  ConsumerState<ScaffoldWithBottomNavBar> createState() =>
      _ScaffoldWithBottomNavBarState();
}

class _ScaffoldWithBottomNavBarState
    extends ConsumerState<ScaffoldWithBottomNavBar> {
  void _tap(int index) {
    ref.read(scaffoldBottomNavControllerProvider.notifier).setPosition(index);
    ref.invalidate(ingredientMultiplierProvider);
```

```
  ref.invalidate(instructionsCounterProvider);
  if (index == 0) {
    context.goNamed(AppRoute.home.name);
    ref.read(recipeControllerProvider.notifier).ref.invalidateSelf();
  } else if (index == 1) {
    context.goNamed(AppRoute.challenges.name);
  } else if (index == 2) {
    context.goNamed(AppRoute.saved.name);
  } else if (index == 3) {
    context.goNamed(AppRoute.friends.name);
  } else {
    context.goNamed(AppRoute.profile.name);
  }
}

@override
Widget build(BuildContext context) {
  int position = ref.watch(scaffoldBottomNavControllerProvider);
  return Scaffold(
    body: widget.child,
    bottomNavigationBar: BottomNavigationBar(
      currentIndex: position,
      onTap: (index) => _tap(index),
      items: [
        _buildBottomNavBarItemWidget(Assets.icons.home, StringConstants.home),
        _buildBottomNavBarItemWidget(
            Assets.icons.challenges, StringConstants.challenges),
        _buildBottomNavBarItemWidget(
            Assets.icons.saved, StringConstants.saved),
        _buildBottomNavBarItemWidget(
```

```
            Assets.icons.friends, StringConstants.friends),
        _buildBottomNavBarItemWidget(
            Assets.icons.profile, StringConstants.profile),
      ],
    ),
  );
 }

 BottomNavigationBarItem _buildBottomNavBarItemWidget(
    String iconName, String label) {
  return BottomNavigationBarItem(
    icon: SvgPicture.asset(
      iconName,
      height: AppConstants.bottomNavbarIconHeight,
    ),
    activeIcon: SvgPicture.asset(
      iconName,
      height: AppConstants.bottomNavbarIconHeight,
      theme: const SvgTheme(
        currentColor: ThemeColors.primary,
      ),
      colorFilter:
          const ColorFilter.mode(ThemeColors.primary, BlendMode.srcATop),
    ),
    label: label);
 }
}
```

Output:-

Hello, Mateo

Prepare delicious meals and win prizes!

Points
0

Search recipes

Search for: chicken

Sweet & Sour Chicken
10 points

Asparagus-Stuffed Chicken Breast
10 points

Low-Carb Mediterranean Chicken And Tomato By
10 points 30 mins

Easy Butter Chicken
10 points 50 mins

Home Challenges Saved Friends Profile

Add friend

Search the community


Marija


Laura


Inga


test

Home Challenges Saved Friends Profile

Saved recipes

Jacket Potato: The Old Fashioned

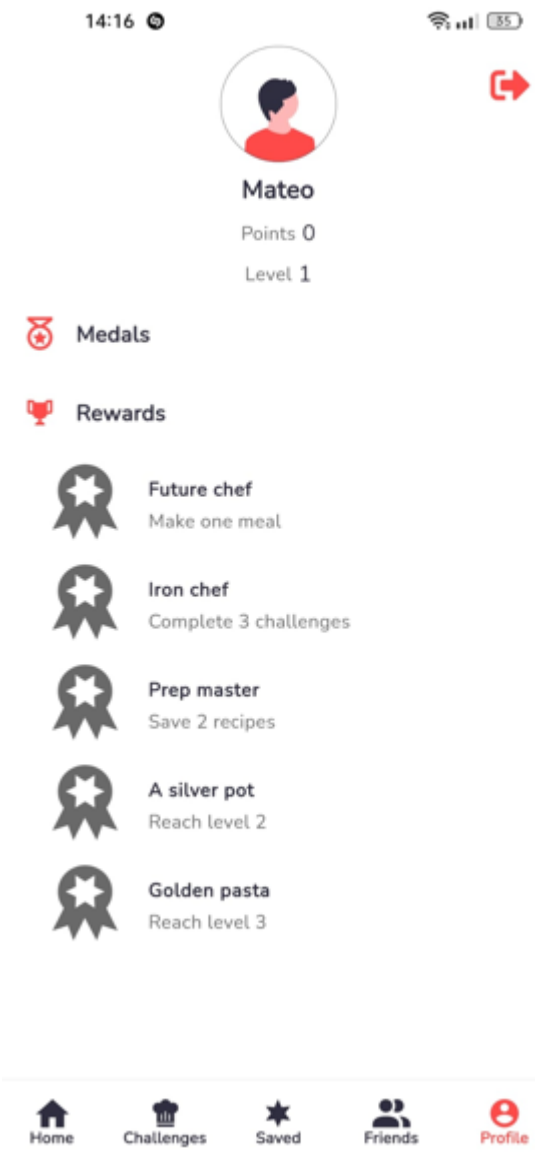
10 points

Vegan Butternut Squash Al Pastor Tacos

10 points

How To Make A Fresh Seacuterie Board

10 points


14:16
Mateo
Points 0
Level 1
Medals
Rewards
Future chef
Make one meal
Iron chef
Complete 3 challenges
Prep master
Save 2 recipes
A silver pot
Reach level 2
Golden pasta
Reach level 3
Home
Challenges
Saved
Friends
Profile

Culinary missions

August 28 - September 3

Beginner

Make a dish with eggs

0 1 10 points

Veggie lover

Use tomato or salad in 3 dishes

0 3 15 points

Chicken master

Use the chicken in 3 dishes

0 3 15 points

Home Challenges Saved Friends Profile

Experiment No.	06
Experiment Title.	To Connect Flutter UI with fireBase database
Roll No.	02
Name	Vanshika Ambwani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO3: Analyze and Build production ready Flutter App by incorporating backend services and deploying on Android / iOS
Grade:	

Experiment No. 6

Aim:-

To Connect Flutter UI with fireBase database

Theory :

Firebase provides a powerful backend solution for Flutter applications, enabling features like authentication, real-time database, and cloud storage. By integrating Firebase, developers can easily manage user authentication and store data without setting up a separate backend. Firebase is a cloud-based platform by Google that provides backend services for mobile and web applications, eliminating the need to manage servers. It integrates seamlessly with Flutter to enhance app functionality, security, and performance.

It includes Authentication (email, social logins, phone sign-in), Cloud Firestore & Realtime Database for real-time data syncing, and Cloud Storage for handling images, videos, and documents. Cloud Messaging (FCM) enables push notifications, while Crashlytics and Performance Monitoring help in debugging and optimizing app performance. Remote Config allows updating app features without requiring updates, and Analytics provides insights into user behavior. With Firebase, Flutter apps can be more scalable, secure, and feature-rich without complex backend management.

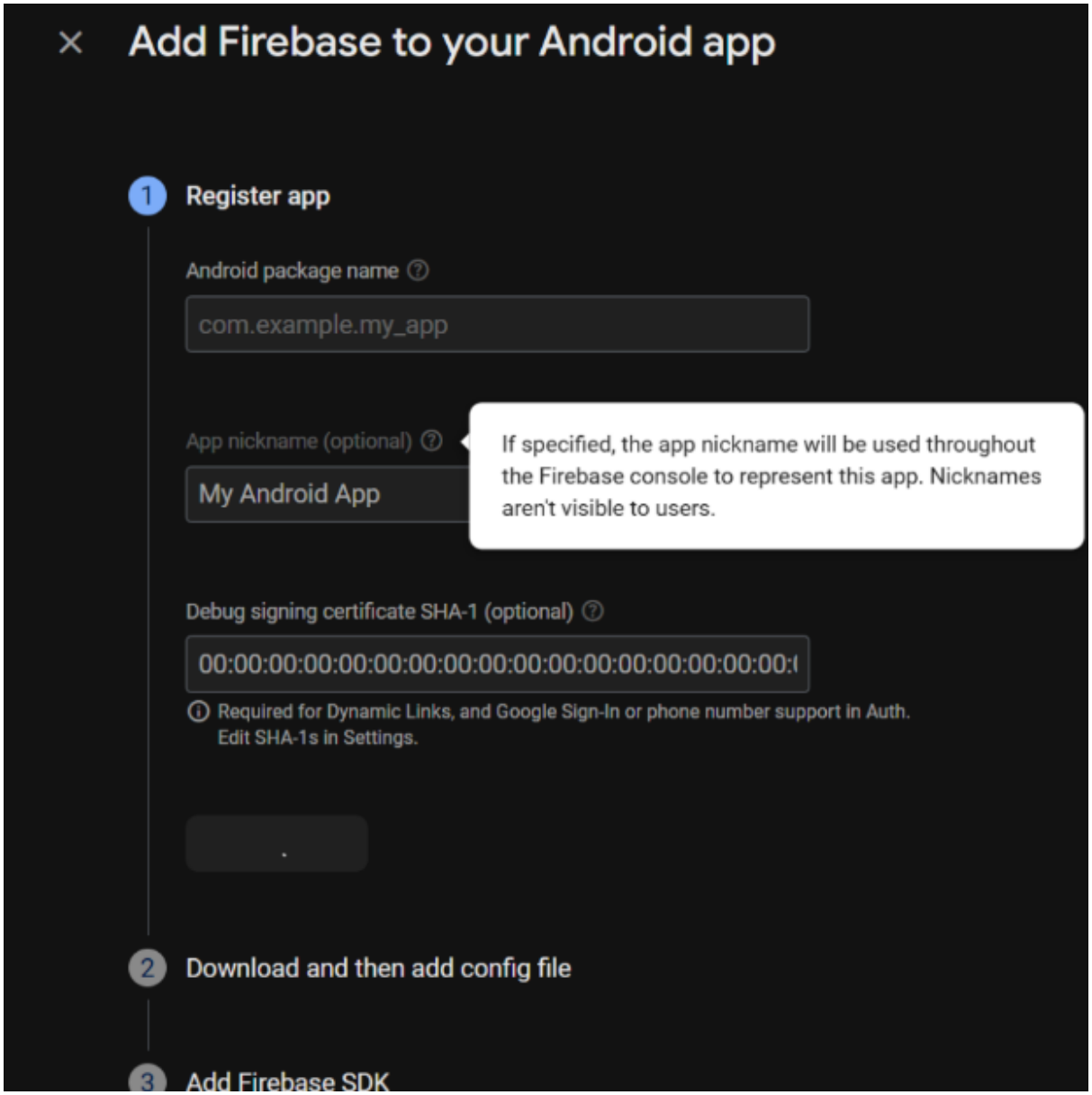
Code:-

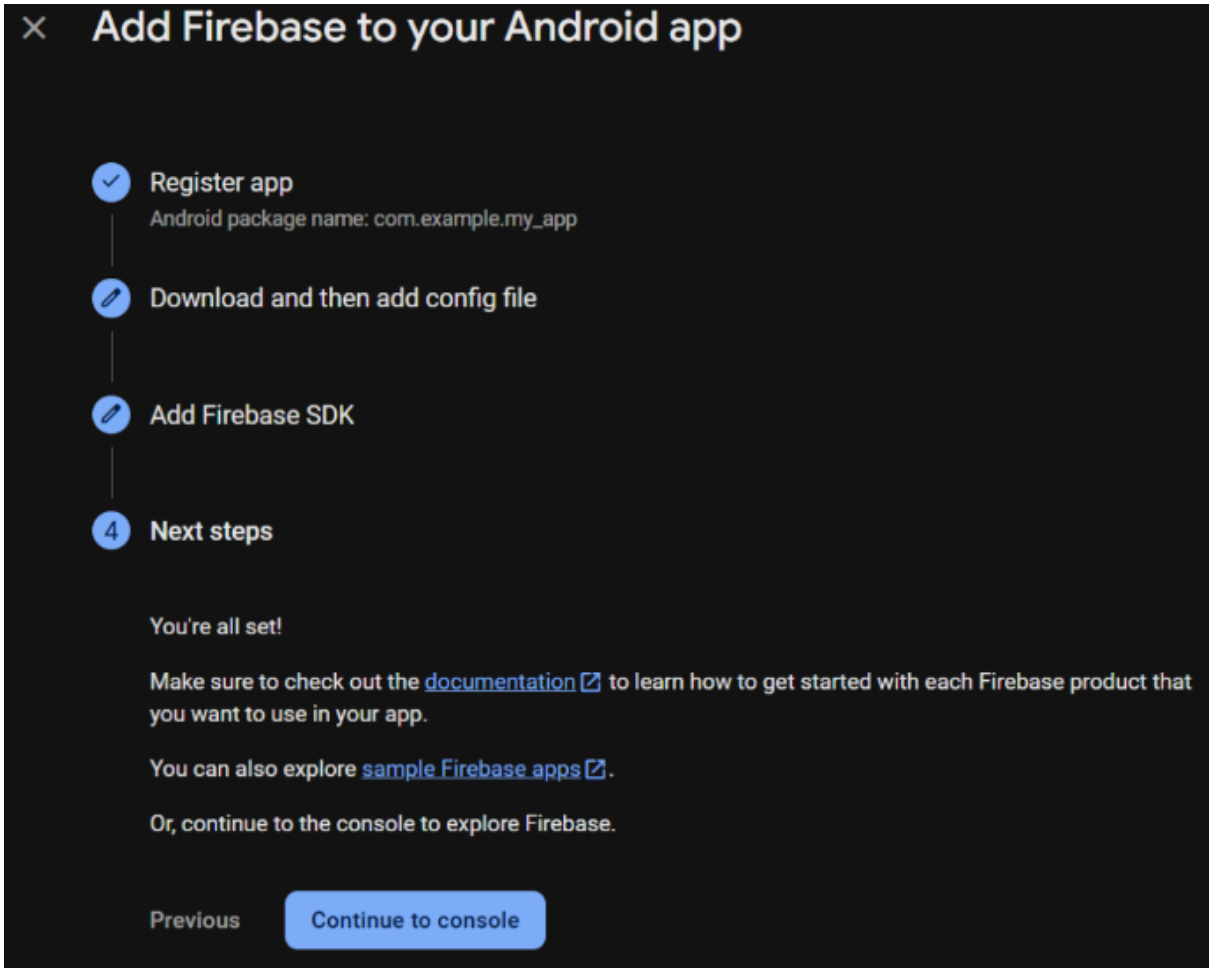
```
// File generated by FlutterFire CLI.

// ignore_for_file: lines_longer_than_80_chars,
avoid_classes_with_only_static_members

import 'package:firebase_core/firebase_core.dart' show FirebaseOptions;

import 'package:flutter/foundation.dart'

    show defaultTargetPlatform, kIsWeb, TargetPlatform;


/// Default [FirebaseOptions] for use with your Firebase apps.

///

/// Example:

/// ```dart

/// import 'firebase_options.dart';

/// // ...

/// await Firebase.initializeApp(

///   options: DefaultFirebaseOptions.currentPlatform,
```

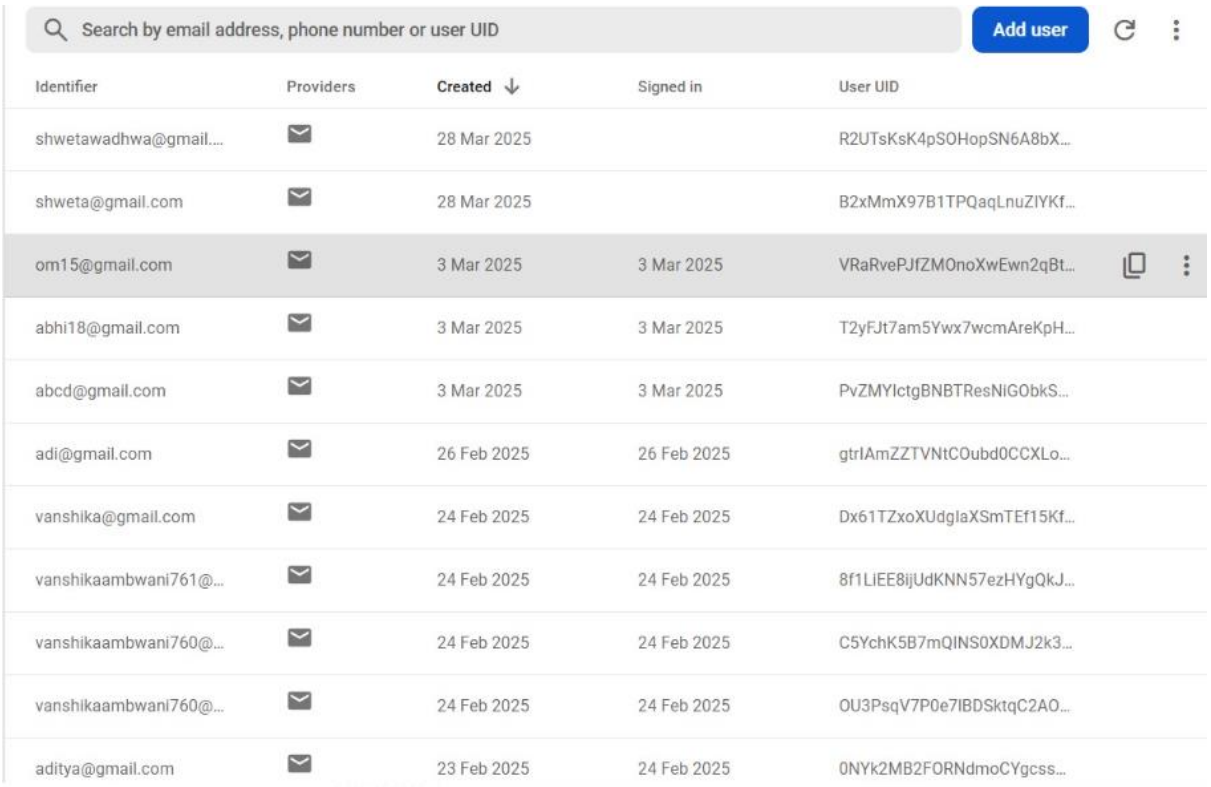
```
/// );
/// ```
class DefaultFirebaseOptions {
  static FirebaseOptions get currentPlatform {
    if (kIsWeb) {
      return web;
    }
    switch (defaultTargetPlatform) {
      case TargetPlatform.android:
        return android;
      case TargetPlatform.iOS:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been configured for ios - '
          'you can reconfigure this by running the FlutterFire CLI again.',
        );
      case TargetPlatform.macOS:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been configured for macos - '
          'you can reconfigure this by running the FlutterFire CLI again.',
        );
      case TargetPlatform.windows:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been configured for windows - '
          'you can reconfigure this by running the FlutterFire CLI again.',
        );
      case TargetPlatform.linux:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been configured for linux - '
          'you can reconfigure this by running the FlutterFire CLI again.',
        );
```

```
      default:
        throw UnsupportedError(
          'DefaultFirebaseOptions are not supported for this platform.',
        );
    }
  }

  static const FirebaseOptions web = FirebaseOptions(
    apiKey: 'AIzaSyBGZ4Uen3ahPFlalRk8ySlAvbRXHfyvH1Y',
    appId: '1:708737726793:web:aab7601652171bb49fc298',
    messagingSenderId: '708737726793',
    projectId: 'foodie-recipe-app',
    authDomain: 'foodie-recipe-app.firebaseapp.com',
    storageBucket: 'foodie-recipe-app.appspot.com',
  );

  static const FirebaseOptions android = FirebaseOptions(
    apiKey: 'AIzaSyCumm_1kNLPnfmI53MIzzzG0N9zRq4X4i0',
    appId: '1:708737726793:android:0bb9e209de2093259fc298',
    messagingSenderId: '708737726793',
    projectId: 'foodie-recipe-app',
    storageBucket: 'foodie-recipe-app.appspot.com',
  );
}
```

Output:-


Add Firebase to your Android app
1
Register app
Android package name
com.example.my_app
App nickname (optional)
My Android App
If specified, the app nickname will be used throughout the Firebase console to represent this app. Nicknames aren't visible to users.
Debug signing certificate SHA-1 (optional)
00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:0
Required for Dynamic Links, and Google Sign-In or phone number support in Auth. Edit SHA-1s in Settings.
2
Download and then add config file
3
Add Firebase SDK


Add Firebase to your Android app
Register app
Android package name: com.example.my_app
Download and then add config file
Add Firebase SDK
4
Next steps
You're all set!
Make sure to check out the documentation to learn how to get started with each Firebase product that you want to use in your app.
You can also explore sample Firebase apps.
Or, continue to the console to explore Firebase.
Previous
Continue to console


Search by email address, phone number or user UID
Add user
Identifier
Providers
Created
Signed in
User UID
shwetawadhwa@gmail....
28 Mar 2025
R2UTsKsK4pSOHopSN6A8bX...
shweta@gmail.com
28 Mar 2025
B2xMmX97B1TPQaqLnuZlYKf...
om15@gmail.com
3 Mar 2025
3 Mar 2025
VRaRvePJfZMOnoXwEwn2qBt...
abhi18@gmail.com
3 Mar 2025
3 Mar 2025
T2yFJt7am5Ywx7wcmAreKpH...
abcd@gmail.com
3 Mar 2025
3 Mar 2025
PvZMYIctgBNBTResNiGObkS...
adi@gmail.com
26 Feb 2025
26 Feb 2025
gtrIAmZZTVNtCOubd0CCXLo...
vanshika@gmail.com
24 Feb 2025
24 Feb 2025
Dx61TZxoXUdgIaXSmTEf15Kf...
vanshikaambwani761@...
24 Feb 2025
24 Feb 2025
8f1LiEE8ijUdKNN57ezHYgQkJ...
vanshikaambwani760@...
24 Feb 2025
24 Feb 2025
C5YchK5B7mQINS0XDMJ2k3...
vanshikaambwani760@...
24 Feb 2025
24 Feb 2025
OU3PsqV7P0e7IBDSktqC2AO...
aditya@gmail.com
23 Feb 2025
24 Feb 2025
0NYk2MB2FORNdmoCYgcss...

Get on a culinary adventure

Existing member

New member

15:42

Foodie

Email

Password

Log in

Don't have a profile? Join us

MAD & PWA Lab

Journal

Experiment No.	07
Experiment Title.	To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.
Roll No.	02
Name	Vanshika Ambwani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO4: Understand various PWA frameworks and their requirements
Grade:	

Experiment No. 7

Aim: To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.

Theory:

- **Regular Web App**

  A regular web app is a website that is designed to be accessible on all mobile devices such that the content gets fit as per the device screen. It is designed using a web technology stack (HTML, CSS, JavaScript, Ruby, etc.) and operates via a browser. They offer various native-device features and functionalities. However, it entirely depends on the browser the user is using. In other words, it might be possible that you can access a native-device feature on Chrome but not on Safari or Mozilla Firefox because the browsers are incompatible with that feature.

- **Progressive Web App**

  Progressive Web App (PWA) is a regular web app, but some extras enable it to deliver an excellent user experience. It is a perfect blend of desktop and mobile application experience to give both platforms to the end-users.

- **Difference between PWAs vs. Regular Web Apps:**

  1. Native Experience: Though a PWA runs on web technologies (HTML, CSS, JavaScript) like a Regular web app, it gives user experience like a native mobile application. It can use most native device features, including push notifications, without relying on the browser or any other entity. It offers a seamless and integrated user experience that it is quite tough for one to differentiate between a PWA and a Native application by considering its look and feel.

  2. Ease of Access: Unlike other mobile apps, PWAs do not demand longer download time and make memory space available for installing the applications. The PWAs can be shared and installed by a link, which cuts down the number of steps to install and use. These applications can easily keep an app icon on the user's home screen, making the app easily accessible to the users and helps the brands remain in the users' minds, and improving the chances of interaction.

  3. Faster Services: PWAs can cache the data and serve the user with text stylesheets, images, and other web content even before the page loads completely. This lowers the waiting time for the end-users and helps the brands improve the user engagement and retention rate, which eventually adds value to their business.

  4. Engaging Approach: As already shared, the PWAs can employ push notifications and other native device features more efficiently. Their interaction does not depend on the browser user uses. This eventually improves the chances of notifying the user regarding your services, offers, and other options related to your brand and keeping them hooked to your brand. In simpler words, PWAs let you maintain the user engagement and retention rate.

5. Updated Real: Time Data Access: Another plus point of PWAs is that these apps get updated on their own. They do not demand the end-users to go to the App Store or other such platforms to download the update and wait until installed.

6. Discoverable: PWAs reside in web browsers. This implies higher chances of optimizing them as per the Search Engine Optimization (SEO) criteria and improving the Google rankings like that in websites and other web apps.

7. Lower Development Cost: Progressive web apps can be installed on the user device like a native device, but it does not demand submission on an App Store. This makes it far more cost-effective than native mobile applications while offering the same set of functionalities.

The main features are:

1. Progressive — They work for every user, regardless of the browser chosen because they are built at the base with progressive improvement principles.
2. Responsive — They adapt to the various screen sizes: desktop, mobile, tablet, or dimensions that can later become available.
3. Updated — Information is always up-to-date thanks to the data update process offered by service workers.
4. Secure — Exposed over HTTPS protocol to prevent the connection from displaying information or altering the contents.
5. Searchable — They are identified as “applications” and are indexed by search engines.
6. Reactivable — Make it easy to reactivate the application thanks to capabilities such as web notifications.
7. Installable — They allow the user to “save” the apps that he considers most useful with the corresponding icon on the screen of his mobile terminal (home screen) without having to face all the steps and problems related to the use of the app store.
8. Linkable — Easily shared via URL without complex installations.
9. Offline — Once more it is about putting the user before everything, avoiding the usual error message in case of weak or no connection. The PWA are based on two particularities: first of all the ‘skeleton’ of the app, which recalls the page structure, even if its contents do not respond and its elements include the header, the page layout, as well as an illustration that signals that the page is loading.

Code:

1. **Manifest.json**

```
{
  "name": "Shopping App",
  "short_name": "MyApp",
  "description": "An online shopping platform with a wide range of
products.",   "start_url": "/",
  "scope": "/",
```

```
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#ff6600",
  "icons": [
   {
    "src": "assets/images/banner-1.png",
    "sizes": "192x192",
    "type": "image/png",
    "purpose": "any maskable"
   },
   {
    "src": "assets/images/banner-2.png",
    "sizes": "512x512",
    "type": "image/png",
    "purpose": "any maskable"
   }
  ],
  "shortcuts": [
   {
    "name": "Shop Now",
    "short_name": "Shop",
    "description": "Go directly to shopping",
    "url": "/index.html",
    "icons": [
     {
      "src": "assets/images/shortcut-icon.png",
      "sizes": "96x96",
      "type": "image/png"
     }
    ]
   }
  ]
}
```

2. **Service-worker.js** self.addEventListener("install", (event) => {

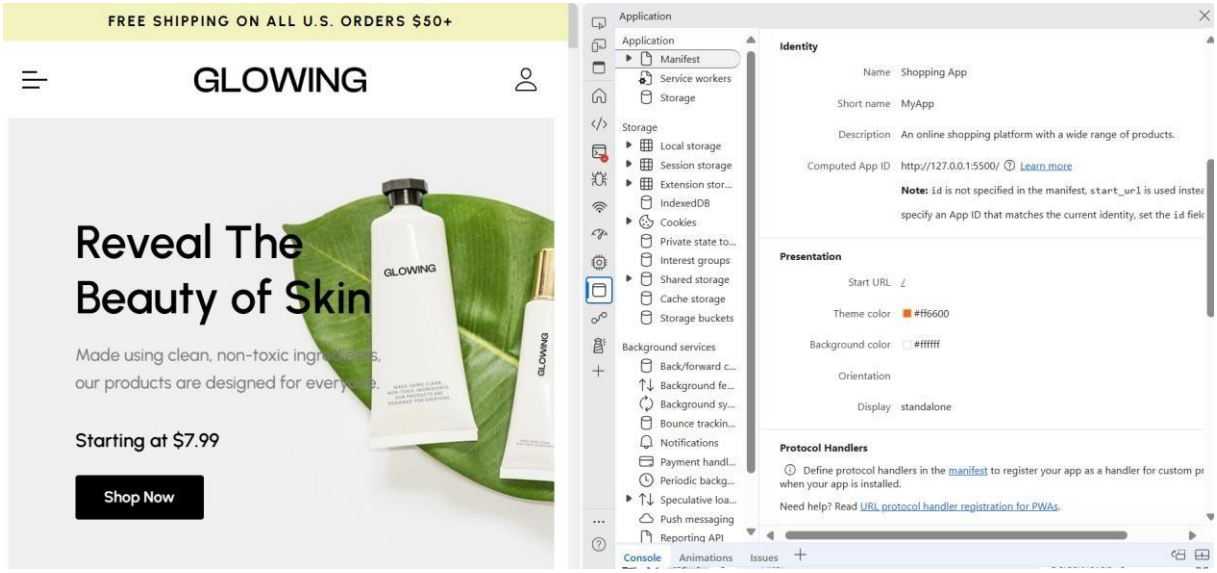
```
console.log("Service Worker Installed");
});

self.addEventListener("fetch", (event) => {
event.respondWith(fetch(event.request));
});
```

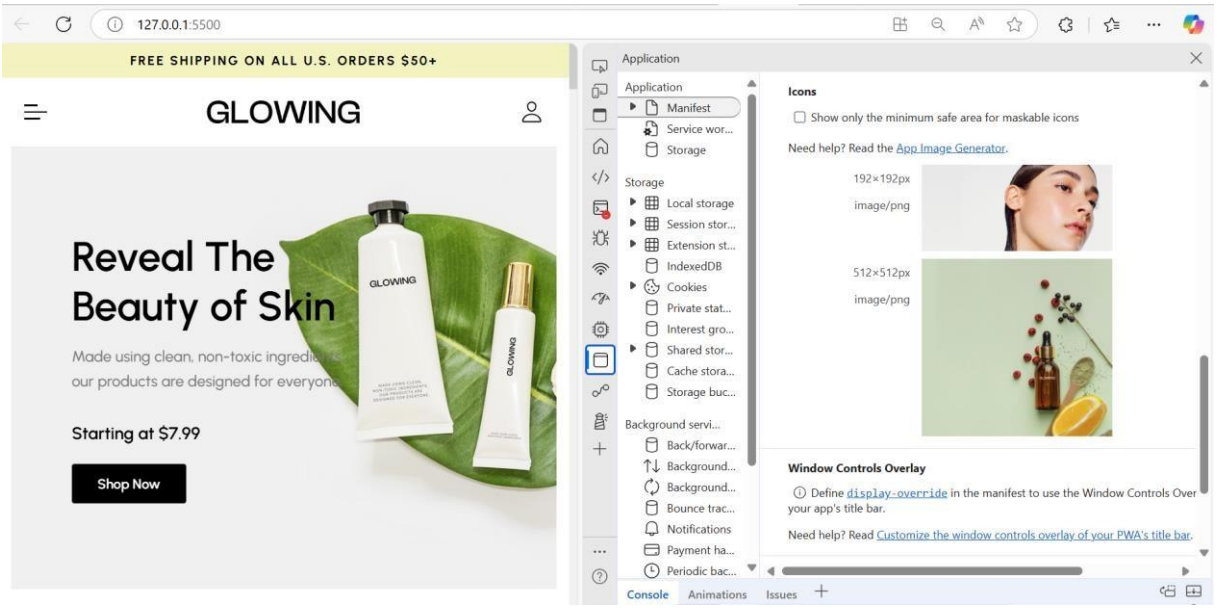
3. **Script.js file:**

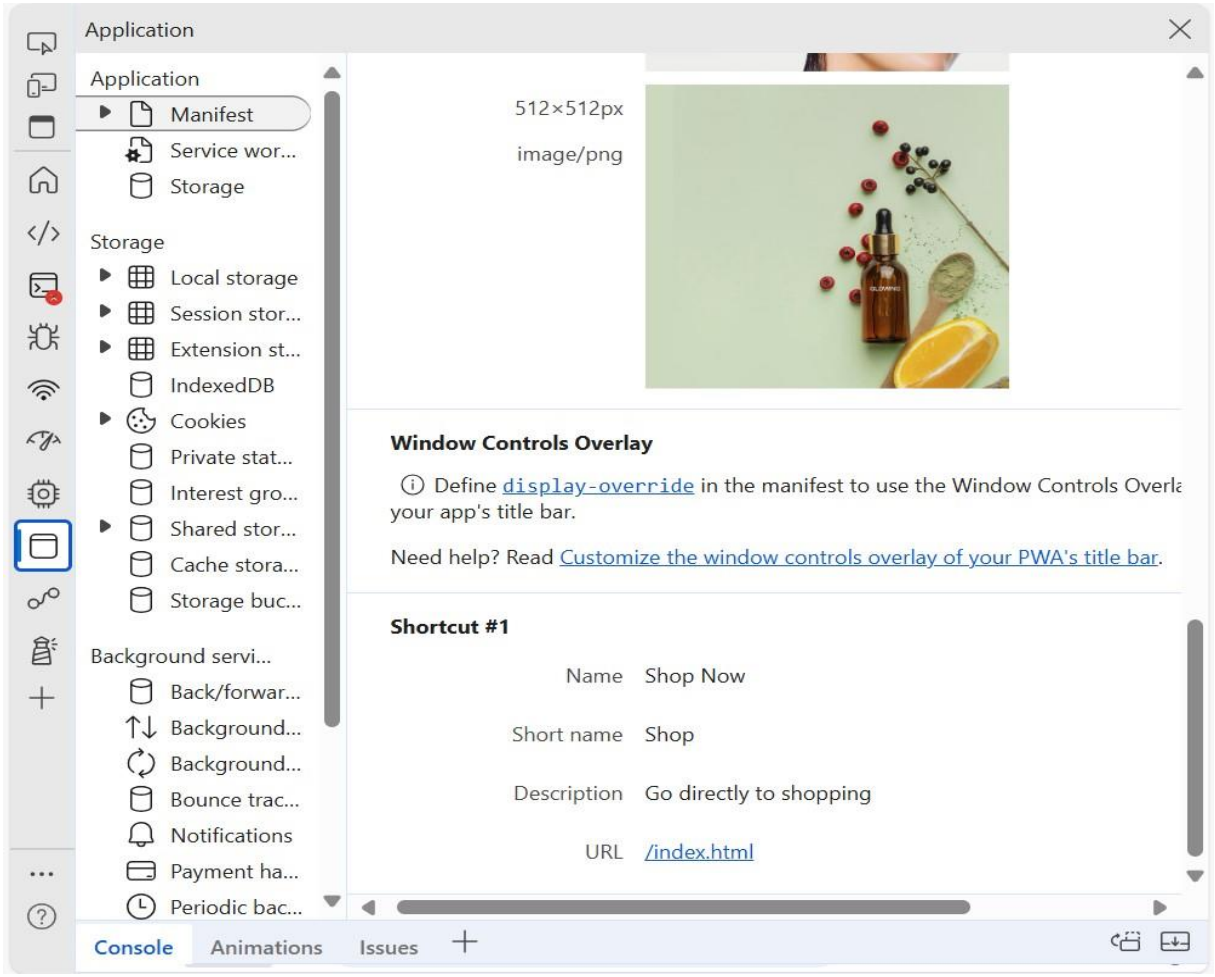
```
'use strict';

/**
 * add event on element
 */

const addEventOnElem = function (elem, type, callback) {
if (elem.length > 1) {     for (let i = 0; i < elem.length; i++) {
elem[i].addEventListener(type, callback);
    }
  } else {
    elem.addEventListener(type, callback);
  }
}


/**
 * navbar toggle
 */

const navTogglers = document.querySelectorAll("[data-nav-toggler]");
const navbar = document.querySelector("[data-navbar]"); const
navbarLinks = document.querySelectorAll("[data-nav-link]"); const
overlay = document.querySelector("[data-overlay]");

const toggleNavbar = function () {  navbar.classList.toggle("active");
  overlay.classList.toggle("active");
}

addEventOnElem(navTogglers, "click", toggleNavbar);

const closeNavbar = function () {  navbar.classList.remove("active");
  overlay.classList.remove("active");
}

addEventOnElem(navbarLinks, "click", closeNavbar);
```

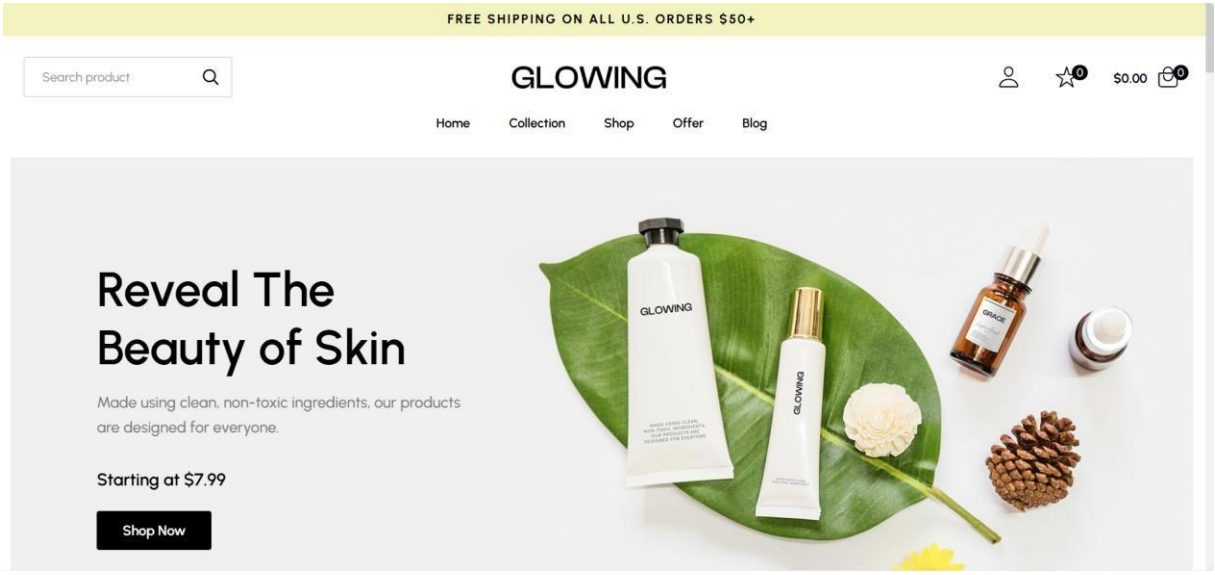
```
/**
 * header sticky & back top btn active
 */

const header = document.querySelector("[data-header]"); const
backTopBtn = document.querySelector("[data-back-top-btn]");

const headerActive = function () {
if (window.scrollY > 150) {
header.classList.add("active");
    backTopBtn.classList.add("active");
  } else {
    header.classList.remove("active");
    backTopBtn.classList.remove("active");
  }
}

addEventOnElem(window, "scroll", headerActive);

let lastScrolledPos = 0;

const headerSticky = function () {     if
(lastScrolledPos >= window.scrollY) {
    header.classList.remove("header-hide");
  } else {
    header.classList.add("header-hide");
  }

  lastScrolledPos = window.scrollY;
}

addEventOnElem(window, "scroll", headerSticky);


/**
 * scroll reveal effect
 */

const sections = document.querySelectorAll("[data-section]");

const scrollReveal = function () {    for
(let i = 0; i < sections.length; i++) {
    if      (sections[i].getBoundingClientRect().top      <      window.innerHeight      /      2)      {
sections[i].classList.add("active");
    }
  }
}

scrollReveal();

addEventOnElem(window, "scroll", scrollReveal);
```

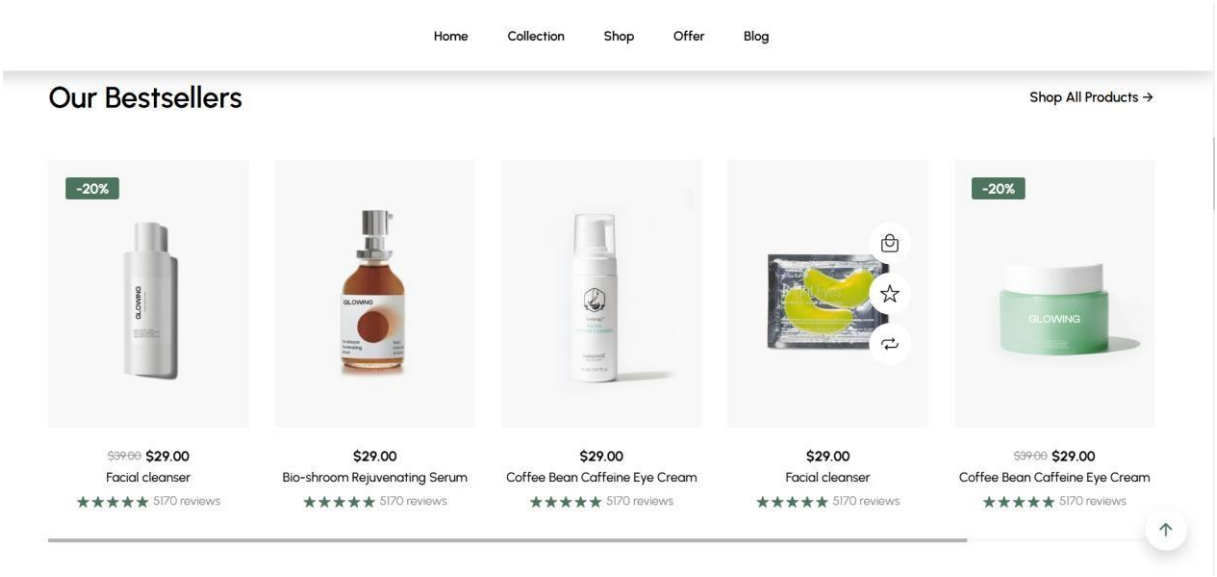
Output:


FREE SHIPPING ON ALL U.S. ORDERS $50+
GLOWING
Reveal The Beauty of Skin
Starting at $7.99
Shop Now
Application
Manifest
Service workers
Storage
Identity
Name Shopping App
Short name MyApp
Description An online shopping platform with a wide range of products.
Presentation
Start URL /
Theme color #ff6600
Background color #ffffff
Orientation
Display standalone
Protocol Handlers


127.0.0.1:5500
FREE SHIPPING ON ALL U.S. ORDERS $50+
GLOWING
Reveal The Beauty of Skin
Starting at $7.99
Shop Now
Application
Manifest
Storage
Icons
Show only the minimum safe area for maskable icons
Need help? Read the App Image Generator.
192×192px
image/png
512×512px
image/png
Window Controls Overlay
Console
Animations
Issues


Application
Application
Manifest
Service wor...
Storage
Storage
Local storage
Session stor...
Extension st...
IndexedDB
Cookies
Private stat...
Interest gro...
Shared stor...
Cache stora...
Storage buc...
Background servi...
Back/forwar...
Background...
Background...
Bounce trac...
Notifications
Payment ha...
Periodic bac...
512×512px
image/png
Window Controls Overlay
Define display-override in the manifest to use the Window Controls Overla your app's title bar.
Need help? Read Customize the window controls overlay of your PWA's title bar.
Shortcut #1
Name Shop Now
Short name Shop
Description Go directly to shopping
URL /index.html
Console
Animations
Issues


FREE SHIPPING ON ALL U.S. ORDERS $50+
Search product
GLOWING
$0.00
Home
Collection
Shop
Offer
Blog
Reveal The Beauty of Skin
Made using clean, non-toxic ingredients, our products are designed for everyone.
Starting at $7.99
Shop Now
GLOWING
GLOWING
GRACE


Home
Collection
Shop
Offer
Blog
Our Bestsellers
Shop All Products →
-20%
$39.00 $29.00
Facial cleanser
5170 reviews
$29.00
Bio-shroom Rejuvenating Serum
5170 reviews
$29.00
Coffee Bean Caffeine Eye Cream
5170 reviews
$29.00
Facial cleanser
5170 reviews
-20%
$39.00 $29.00
Coffee Bean Caffeine Eye Cream
5170 reviews

Vanshika Ambwani (02), Gunjan Chandnani (06), Akruti Dabas (11)

MAD & PWA Lab Journal

Experiment No.	08
Experiment Title.	To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA
Roll No.	02
Name	Vanshika Ambwani
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

Aim: To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA.

Vanshika Ambwani (02), Gunjan Chandnani (06), Akruti Dabas (11)

Theory:

Service Worker

Service Worker is a script that works on browser background without user interaction independently. Also, It resembles a proxy that works on the user side. With this script, you can track network traffic of the page, manage push notifications and develop “offline first” web applications with Cache API.

Things to note about Service Worker:

- A service worker is a programmable network proxy that lets you control how network requests from your page are handled.
- Service workers only run over HTTPS. Because service workers can intercept network requests and modify responses, "man-in-the-middle" attacks could be very bad.
- The service worker becomes idle when not in use and restarts when it's next needed. You cannot rely on a global state persisting between events. If there is information that you need to persist and reuse across restarts, you can use IndexedDB databases.

What can we do with Service Workers?

- You can dominate **Network Traffic**
  You can manage all network traffic of the page and do any manipulations. For example, when the page requests a CSS file, you can send plain text as a response or when the page requests an HTML file, you can send a png file as a response. You can also send a true response too.

- You can **Cache**
  You can cache any request/response pair with Service Worker and Cache API and you can access these offline content anytime.

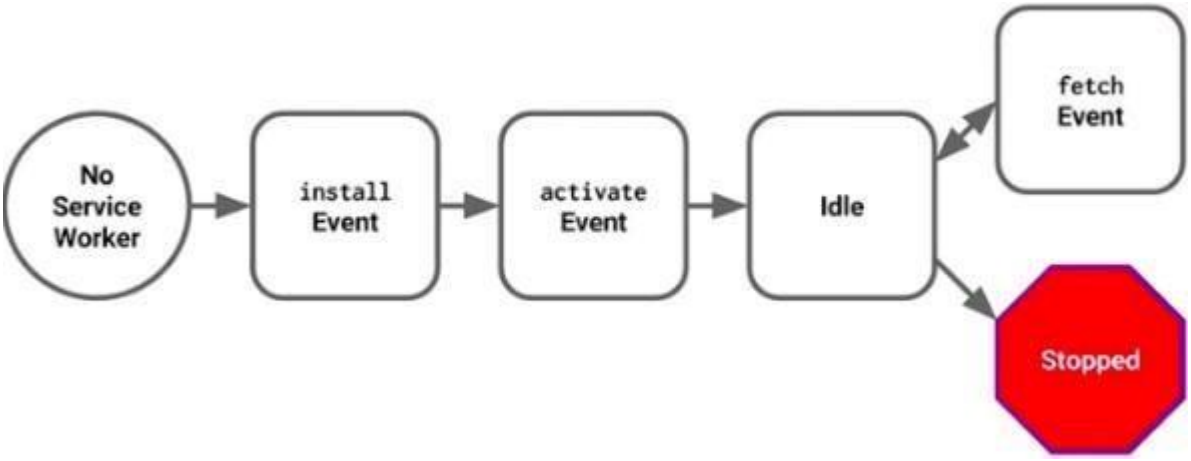
- You can manage **Push Notifications**
  You can manage push notifications with Service Worker and show any information message to the user.
- You can **Continue**
  Although Internet connection is broken, you can start any process with Background Sync of Service Worker.

What can’t we do with Service Workers?

- You can’t access the **Window**
  You can’t access the window, therefore, You can’t manipulate DOM elements. But, you can communicate to the window through post Message and manage processes that you want.

- You can’t work it on **80 Port**
  Service Worker just can work on HTTPS protocol. But you can work on localhost during development.

Service Worker Cycle



A service worker goes through three steps in its life cycle:

- Registration
- Installation
- Activation

Registration

To install a service worker, you need to register it in your main JavaScript code. Registration tells the browser where your service worker is located, and to start installing it in the background. Let's look at an example:

main.js

```
if ('serviceWorker' in navigator) {
  navigator.serviceWorker.register('/service-worker.js')
  .then(function(registration) { console.log('Registration successful,
    scope is:', registration.scope);
  })
  .catch(function(error) { console.log('Service worker
  registration failed, error:', error); });
}
```

This code starts by checking for browser support by examining **navigator.serviceWorker**. The service worker is then registered with navigator.serviceWorker.register, which returns a promise that resolves when the service worker has been successfully registered. The scope of the service worker is then logged with registration.scope. If the service worker is already installed, navigator.serviceWorker.register returns the registration object of the currently active service worker.

The scope of the service worker determines which files the service worker controls, in other words, from which path the service worker will intercept requests. The default scope is the location of the service worker file, and

extends to all directories below. So if service-worker.js is located in the root directory, the service worker will control requests from all files at this domain.

You can also set an arbitrary scope by passing in an additional parameter when registering. For example:

main.js

```
navigator.serviceWorker.register('/service-worker.js', {
  scope: '/app/'
});
```

In this case we are setting the scope of the service worker to /app/, which means the service worker will control requests from pages like /app/, /app/lower/ and /app/lower/lower, but not from pages like /app or /, which are higher.

If you want the service worker to control higher pages e.g. /app (without the trailing slash) you can indeed change the scope option, but you'll also need to set the Service-Worker-Allowed HTTP Header in your server config for the request serving the service worker script.

main.js

```
navigator.serviceWorker.register('/app/service-worker.js', {
  scope: '/app'
});
```

Installation

Once the browser registers a service worker, installation can be attempted. This occurs if the service worker is considered to be new by the browser, either because the site currently doesn't have a registered service worker, or because there is a byte difference between the new service worker and the previously installed one.

A service worker installation triggers an install event in the installing service worker. We can include an install event listener in the service worker to perform some task when the service worker installs. For instance, during the install, service workers can precache parts of a web app so that it loads instantly the next time a user opens it (see caching the application shell). So, after that first load, you're going to benefit from instant repeat loads and your time to interactivity is going to be even better in those cases. An example of an installation event listener looks like this:

service-worker.js

```
// Listen for install event, set callback
self.addEventListener('install', function(event) {
// Perform some task
```

```
});
```

Activation

Once a service worker has successfully installed, it transitions into the activation stage. If there are any open pages controlled by the previous service worker, the new service worker enters a waiting state. The new service worker only activates when there are no longer any pages loaded that are still using the old service worker. This ensures that only one version of the service worker is running at any given time.

When the new service worker activates, an activate event is triggered in the activating service worker. This event listener is a good place to clean up outdated caches (see the Offline Cookbook for an example).

service-worker.js

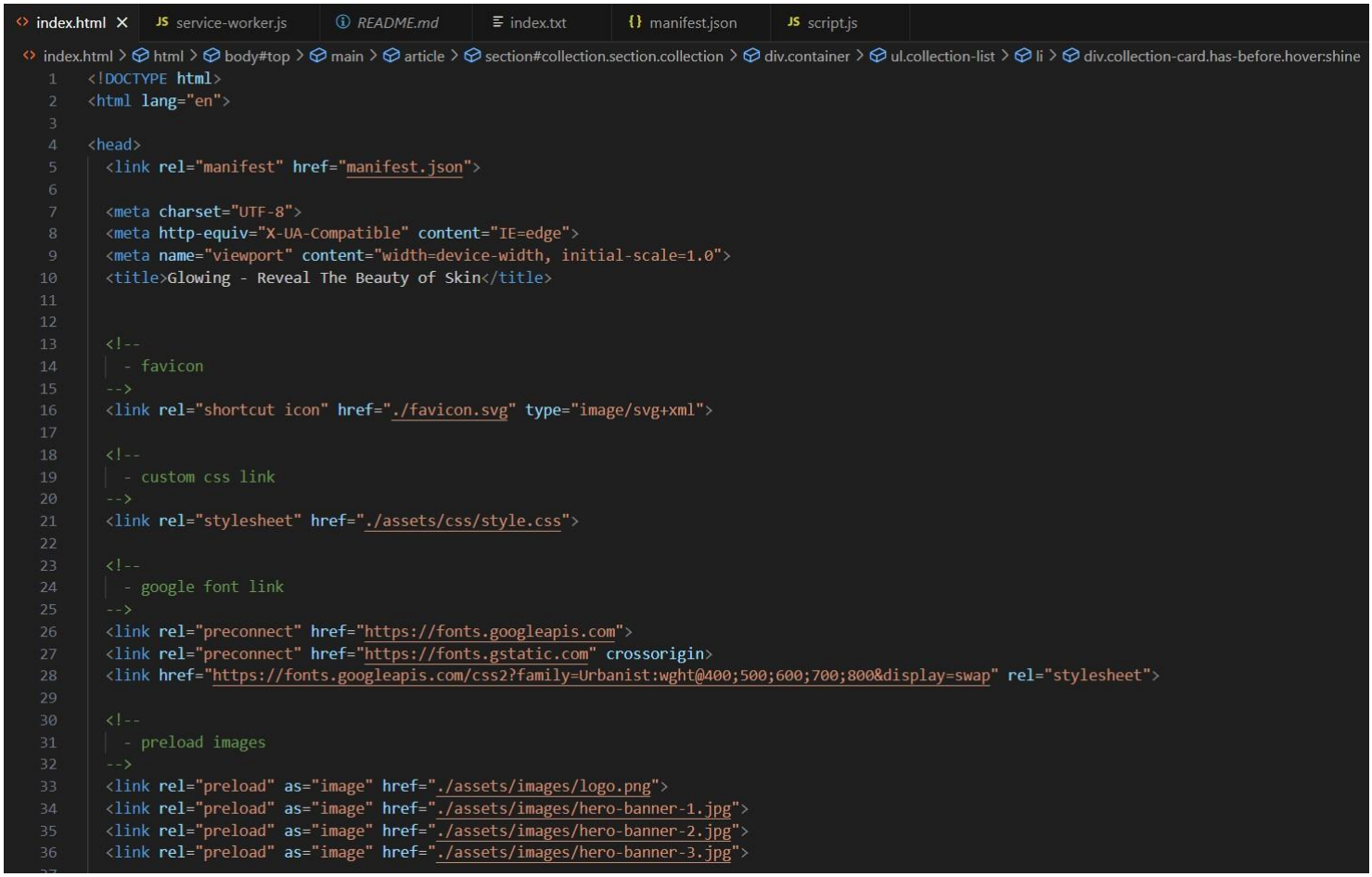
```
self.addEventListener('activate', function(event) { //
  Perform some task
});
```

Once activated, the service worker controls all pages that load within its scope, and starts listening for events from those pages. However, pages in your app that were loaded before the service worker activation will not be under service worker control. The new service worker will only take over when you close and reopen your app, or if the service worker calls **clients.claim()**. Until then, requests from this page will not be intercepted by the new service worker. This is intentional as a way to ensure consistency in your site.

Code

index.html

Vanshika Ambwani (02), Gunjan Chandnani (06), Akruti Dabas (11)



service-worker.js

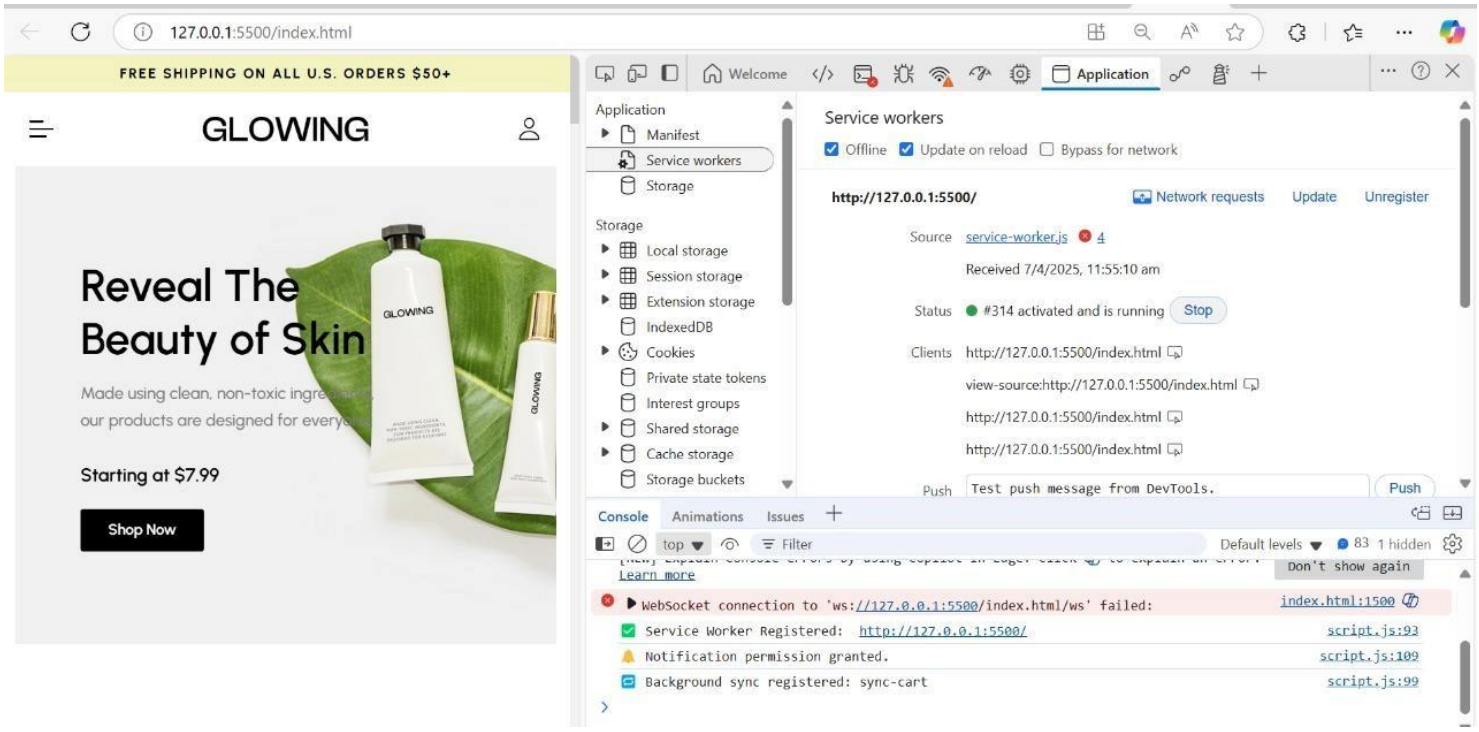
```
self.addEventListener("install", (event) => {
    console.log("Service Worker Installed");
  });

  self.addEventListener("fetch", (event) => {
    event.respondWith(fetch(event.request));
  });
```

manifest.json

```
{
  "name": "Shopping App",
  "short_name": "MyApp",
  "description": "An online shopping platform with a wide range of products.",
  "start_url": "/",
  "scope": "/",
```

```
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#ff6600",
  "icons": [
    {
      "src": "assets/images/banner-1.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any maskable"
    },
    {
      "src": "assets/images/banner-2.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ],
  "shortcuts": [
    {
      "name": "Shop Now",
      "short_name": "Shop",
      "description": "Go directly to shopping",
      "url": "/index.html",
      "icons": [
        {
          "src": "assets/images/shortcut-icon.png",
          "sizes": "96x96",
          "type": "image/png"
        }
      ]
    }
  ]
}
```

Vanshika Ambwani (02), Gunjan Chandnani (06), Akruti Dabas (11)



MAD & PWA Lab Journal

Experiment No.	09
Experiment Title.	To implement Service worker events like fetch, sync and push for E-commerce PWA
Roll No.	02
Name	Vanshika Ambwani
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

EXPERIMENT NO. 9

Aim: To implement Service worker events like fetch, sync and push for E-commerce PWA.

Theory:

Service Worker

Service Worker is a script that works on browser background without user interaction independently. Also, It resembles a proxy that works on the user side. With this script, you can track network traffic of the page, manage push notifications and develop “offline first” web applications with Cache API.

Things to note about Service Worker:

- A service worker is a programmable network proxy that lets you control how network requests from your page are handled.
- Service workers only run over HTTPS. Because service workers can intercept network requests and modify responses, "man-in-the-middle" attacks could be very bad.
- The service worker becomes idle when not in use and restarts when it's next needed. You cannot rely on a global state persisting between events. If there is information that you need to persist and reuse across restarts, you can use IndexedDB databases.
- Service workers make extensive use of promises, so if you're new to promises, then you should stop reading this and check out Promises, an introduction.

Fetch Event

You can track and manage page network traffic with this event. You can check existing cache, manage “cache first” and “network first” requests and return a response that you want.

Of course, you can use many different methods but you can find in the following example a “cache first” and “network first” approach. In this example, if the request’s and current location’s origin are the same (Static content is requested.), this is called “cacheFirst” but if you request a targeted external URL, this is called “networkFirst”.

- **CacheFirst** - In this function, if the received request has cached before, the cached response is returned to the page. But if not, a new response requested from the network.
- **NetworkFirst** - In this function, firstly we can try getting an updated response from the network, if this process completed successfully, the new response will be cached and returned. But if this process fails, we check whether the request has been cached before or not. If a cache exists, it is returned to the page, but if not, this is up to you. You can return dummy content or information messages to the page.

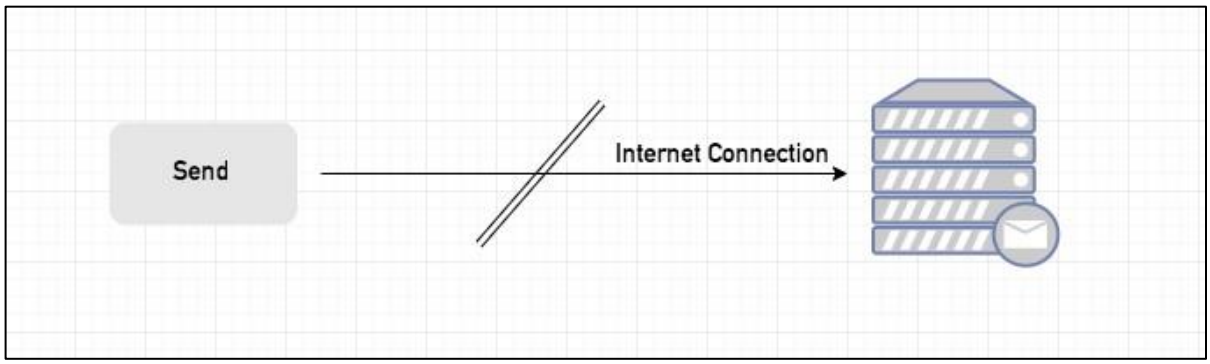
```
self.addEventListener("fetch", function (event) {
    const req = event.request;
    const url = new URL(req.url);

    if (url.origin === location.origin) {
        event.respondWith(cacheFirst(req));
    }
    else {
        event.respondWith(networkFirst(req));
    }
});

async function cacheFirst(req) {
    return await caches.match(req) || fetch(req);
}

async function networkFirst(req) {
    const cache = await caches.open("pwa-dynamic");
    try {
        const res = await fetch(req);
        cache.put(req, res.clone());
        return res;
    } catch (error) {
        const cachedResponse = await cache.match(req);
        return cachedResponse || await caches.match("./noconnection.json");
    }
}
```

Sync Event

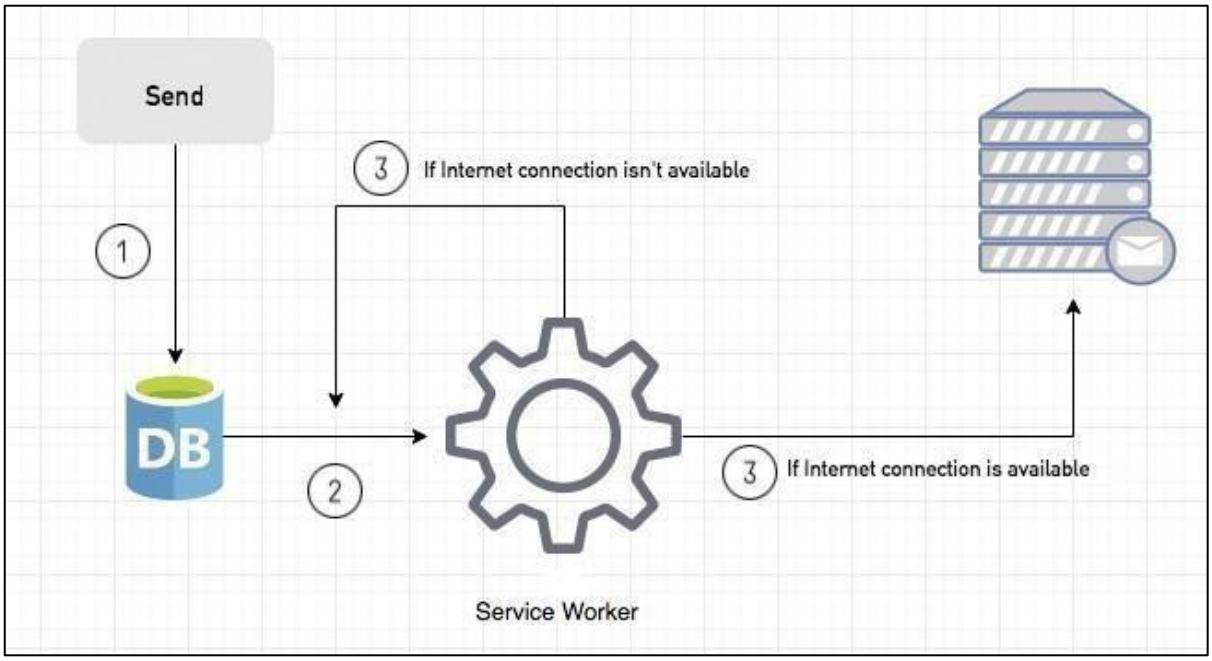
Background Sync is a Web API that is used to delay a process until the Internet connection is stable. We can adapt this definition to the real world; there is an e-mail client application that works on the browser and we want to send an email with this tool. Internet connection is broken while we are writing e-mail content and we didn't realize it. When completing the writing, we click the send button.

Here is a job for the Background Sync.

The following view shows the classical process of sending email to us. If the Internet Connection is broken, we can't send any content to Mail Server.



Here, you can create any scenario for yourself. A sample is in the following for this case.



1. When we click the “send” button, email content will be saved to IndexedDB.
2. Background Sync registration.
3. **If the Internet connection is available**, all email content will be read and sent to Mail Server.

If the Internet connection is unavailable, the service worker waits until the connection is available even though the window is closed. When it is available, email content will be sent to Mail Server.

You can see the working process within the following code block.

Event Listener for Background Sync Registration

```
document.querySelector("button").addEventListener("click", async () => {
    var swRegistration = await navigator.serviceWorker.register("sw.js");
    swRegistration.sync.register("helloSync").then(function () {
        console.log("helloSync success [main.js]");
    });
});
```

Event Listener for sw.js

```
self.addEventListener('sync', event => {
    if (event.tag == 'helloSync') {
        console.log("helloSync [sw.js]");
    }
});
```

Push Event

This is the event that handles push notifications that are received from the server. You can apply any method with received data.

We can check in the following example.
“Notification.requestPermission();” is the necessary line to show notification to the user. If you don’t want to show any notification, you don’t need this line.

In the following code block is in sw.js file. You can handle push notifications with this event. In this example, I kept it simple. We send an object that has “method” and “message” properties. If the method value is “pushMessage”, we open the information notification with the “message” property.

```
self.addEventListener('push', event => {
    if (event && event.data) {
        var data = event.data.json();
        if (data.method === "pushMessage") {
            event.waitUntil(self.registration.showNotification("Test App", {
                body: data.message
            }));
        }
    }
});
```

You can use Application Tab from Chrome Developer Tools for testing push notification.

Code:

1. Push Notification code:

```
self.addEventListener("install", (event) => {
  console.log("✅ Service Worker Installed");
});

self.addEventListener("fetch", (event) => {
  event.respondWith(fetch(event.request));
});

// ✅ Background Sync Event
self.addEventListener("sync", (event) => {
console.log("🔁 Sync Event Triggered:", event.tag);   if
(event.tag === "sync-cart") {
event.waitUntil(syncCartData());
  }
});

async function syncCartData() {
  try {
    console.log("🛒 Syncing cart data...");
    // Add your actual sync logic here
  } catch (err) {
    console.error("❌ Failed to sync cart data:", err);
  }
}

self.addEventListener("message",  (event)  =>  {       if
(event.data && event.data.type === "trigger-push") {
   const { title, body } = event.data;     if
(Notification.permission === "granted") {
self.registration.showNotification(title, {
body,
       icon: "/images/banner-1.png",
       badge: "/images/"
     });
   }
  }
});

// ✅ Push Notification Event self.addEventListener("push",
(event) => {
  console.log("📲 Push Event Received");

  let data = {};   if
(event.data) {
    data = event.data.json();
```

```
  }

  const title = data.title || "New Notification!";
  const options = {
    body: data.body || "You have a new message.",
    icon: "/images/logo.png",
    badge: "/images/logo.png"
  };

  // Check permission before showing notification
if (Notification.permission === "granted") {
event.waitUntil(
      self.registration.showNotification(title, options)
    );
  } else {
    console.warn("⃠ Notification not shown. Permission not granted.");
  }
});
```

2. Fetch and Sync Code:

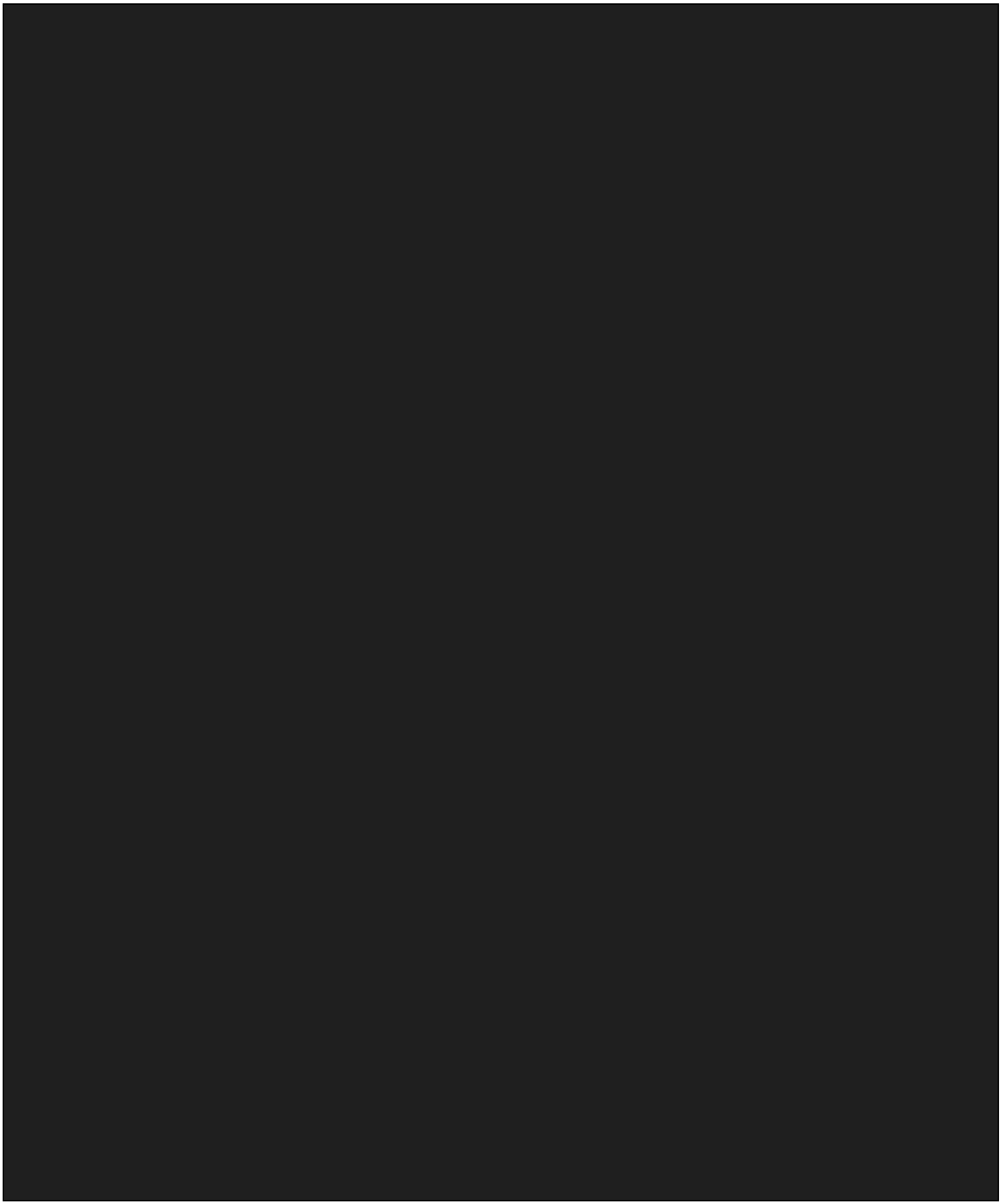
```
'use strict';

    /**
    * Add event on element
    */
    const addEventOnElem = function (elem, type, callback) {
    if (elem.length > 1) {
    for (let i = 0; i < elem.length; i++) {       elem[i].addEventListener(type, callback);
    }
    } else {
    elem.addEventListener(type, callback);
    } };

    /**
    * Navbar toggle
    */
    const navTogglers = document.querySelectorAll("[data-nav-toggler]");
    const navbar = document.querySelector("[data-navbar]");
    const navbarLinks = document.querySelectorAll("[data-nav-link]");
    const overlay = document.querySelector("[data-overlay]");
    const toggleNavbar = function () {
    navbar.classList.toggle("active");
    overlay.classList.toggle("active"); };

addEventOnElem(navTogglers, "click", toggleNavbar);

    const closeNavbar = function () {
    navbar.classList.remove("active");
    overlay.classList.remove("active"); };

addEventOnElem(navbarLinks, "click", closeNavbar);

    /**
    * Header sticky & back top btn active
    */
    const header = document.querySelector("[data-header]");
    const backTopBtn = document.querySelector("[data-back-top-btn]");
    const headerActive = function () {
    if (window.scrollY > 150) {
    header.classList.add("active");
```

```
49.    backTopBtn.classList.add("active"); 50.   } else {
```

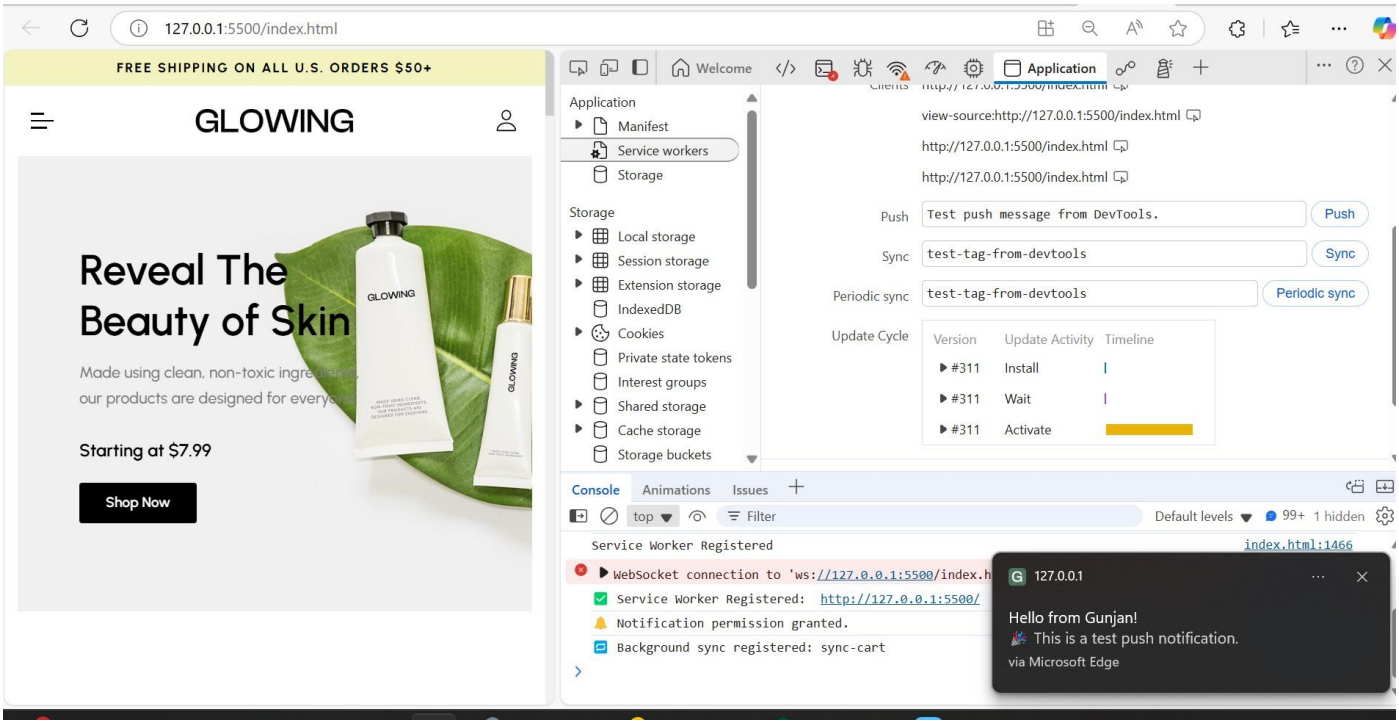
```
    header.classList.remove("active");
    backTopBtn.classList.remove("active");
    } }; 

addEventOnElem(window, "scroll",
headerActive);   

    const headerSticky = function () {
    if (lastScrolledPos >= window.scrollY) {
    header.classList.remove("header-hide");
    } else {
    header.classList.add("header-hide");    }

   lastScrolledPos = window.scrollY; 
};

addEventOnElem(window, "scroll", headerSticky);

    /**
    * Scroll reveal effect
    */
    const sections = document.querySelectorAll("[data-section]");  
    const scrollReveal = function () {
    for (let i = 0; i < sections.length; i++) {
    if (sections[i].getBoundingClientRect().top < window.innerHeight / 2) {
    sections[i].classList.add("active");
    }
    } }; 

    scrollReveal();
    addEventOnElem(window, "scroll", scrollReveal);  
    /**
    * ✅ Register Service Worker and Notification Permission   */
    if ('serviceWorker' in navigator && 'Notification' in window) {
    window.addEventListener("load", () => {
    navigator.serviceWorker.register('/service-worker.js')  
.then((reg) => {
        console.log("✅ Service Worker Registered: ", reg.scope);

              // Optional: Register background sync
              if ('sync' in reg) {
              reg.sync.register('sync-cart')
```

```
100.             .then(() => {
```

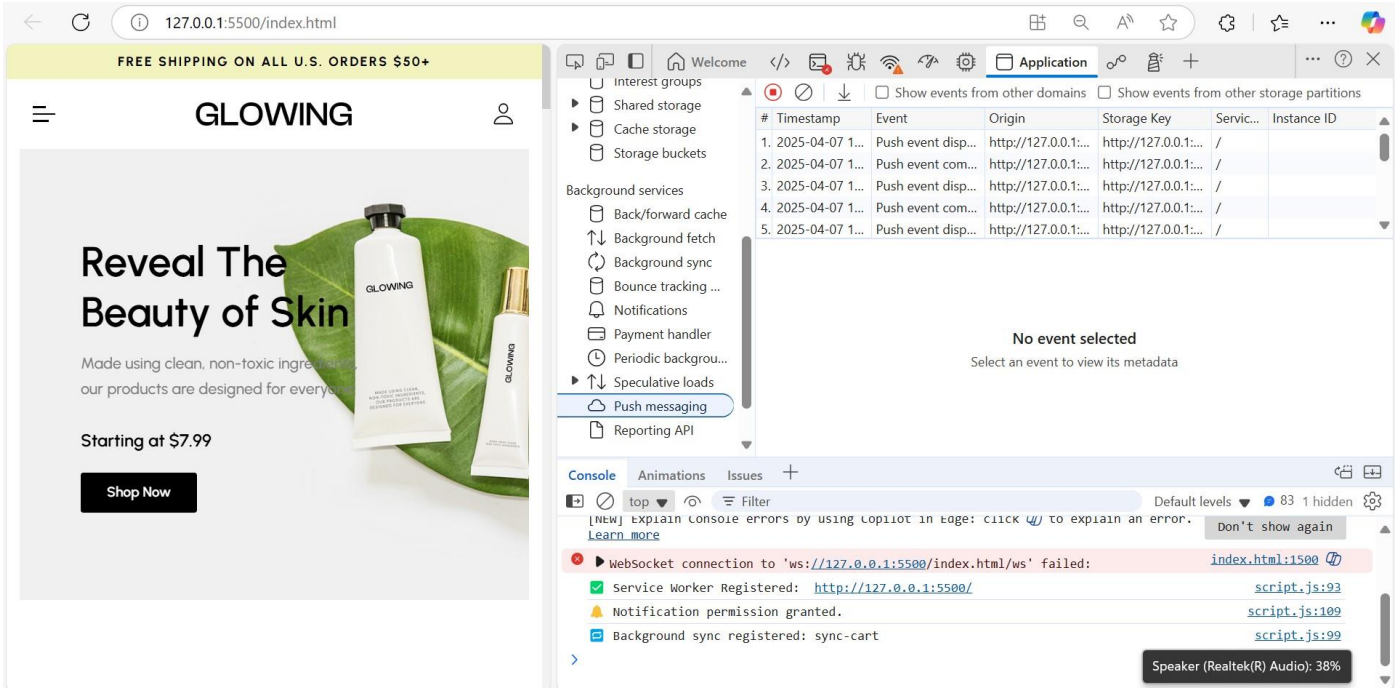
```
            console.log("🔁 Background sync registered: sync-cart");
})
.catch((err) => {
console.error("❌ Sync registration failed:", err);
}); } 
// Ask for notification permission
Notification.requestPermission().then((permission) => {
if (permission === "granted") {
console.log("🔔 Notification permission granted.");
// Optional: Trigger test push manually
setTimeout(() => {
reg.active.postMessage({
type: 'trigger-push',
title: 'Hello from Gunjan!',
body: '🚀 This is a test push notification.'
});
}, 3000); // Trigger after 3s
} else {
console.warn("🔕 Notification permission denied.");
}
});
})
.catch((err) => {
console.error("❌ Service Worker registration failed:", err);
});
}
```



Output:


127.0.0.1:5500/index.html
FREE SHIPPING ON ALL U.S. ORDERS $50+
GLOWING
Reveal The Beauty of Skin
Starting at $7.99
Shop Now
Welcome
Application
Application
Manifest
Service workers
Storage
Storage
Local storage
Session storage
Extension storage
IndexedDB
Cookies
Private state tokens
Interest groups
Shared storage
Cache storage
Storage buckets
view-source:http://127.0.0.1:5500/index.html
http://127.0.0.1:5500/index.html
http://127.0.0.1:5500/index.html
Push
Test push message from DevTools.
Push
Sync
test-tag-from-devtools
Sync
Periodic sync
test-tag-from-devtools
Periodic sync
Update Cycle
Version
Update Activity
Timeline
#311 Install
#311 Wait
#311 Activate
Console
Animations
Issues
top
Filter
Default levels
99+ 1 hidden
Service Worker Registered
index.html:1466
WebSocket connection to 'ws://127.0.0.1:5500/index.h
Service Worker Registered: http://127.0.0.1:5500/
Notification permission granted.
Background sync registered: sync-cart
127.0.0.1
Hello from Gunjan!
This is a test push notification.
via Microsoft Edge


Service Worker Registered: http://127.0.0.1:5500/ script.js:93
Notification permission granted. script.js:109
Background sync registered: sync-cart script.js:99


127.0.0.1:5500/index.html
FREE SHIPPING ON ALL U.S. ORDERS $50+
GLOWING
Reveal The Beauty of Skin
Starting at $7.99
Shop Now
Welcome
Application
Show events from other domains
Show events from other storage partitions
Shared storage
Cache storage
Storage buckets
Background services
Back/forward cache
Background fetch
Background sync
Bounce tracking ...
Notifications
Payment handler
Periodic backgrou...
Speculative loads
Push messaging
Reporting API
Timestamp Event Origin Storage Key Servic... Instance ID
No event selected
Select an event to view its metadata
Console
Animations
Issues
top
Filter
Default levels
83 1 hidden
Don't show again
Learn more
WebSocket connection to 'ws://127.0.0.1:5500/index.html/ws' failed:
index.html:1500
Service Worker Registered: http://127.0.0.1:5500/
script.js:93
Notification permission granted.
script.js:109
Background sync registered: sync-cart
script.js:99
Speaker (Realtek(R) Audio): 38%

MAD & PWA Lab
Journal

Experiment No.	10
Experiment Title.	To study and implement deployment of Ecommerce PWA to GitHub Pages.
Roll No.	02
Name	Vanshika Ambwani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

EXPERIMENT NO: - 10

Name:- Gunjan Chandnani/ Akruti Dabas/ Vanshika Ambwani **Class:-** D15A

Roll:No: - 06/ 11 /02

AIM: - To study and implement deployment of Ecommerce PWA to GitHub Pages.

Theory: -

GitHub Pages: Static Website Hosting Made Simple

GitHub Pages is a free hosting service that allows users to publish **public webpages directly from a GitHub repository**. It is particularly useful for **static websites, project documentation, and blogs**.

Key Features

- **Jekyll Integration**: Built-in support for Jekyll enables easy blogging.
- **Custom Domains**: Allows users to configure their own URLs.
- **Automatic Page Deployment**: Simply push your changes to the repository, and the updates go live.

Why Choose GitHub Pages?

- **Completely Free**: No hosting charges.
- **Seamless GitHub Integration**: Works directly with your repositories.
- **Quick Setup**: Just create a repository, push your files, and your site is live.

Who Uses GitHub Pages?

Companies like **Lyft, CircleCI, and HubSpot** use GitHub Pages for their documentation and static sites. It is widely adopted, appearing in **775 company stacks and 4,401 developer stacks**.

Pros & Cons

Pros

- Familiar interface for GitHub users.

- Simple deployment via the gh-pages branch.
- Supports custom domains with easy DNS configuration.

Cons

- Repositories need to be public unless you have a paid plan.
- Limited HTTPS support for custom domains (expected to improve).
- Jekyll plugins have limited support.

Firebase: A Full-Featured Real-Time Backend

Firebase is a cloud-based **real-time application platform** developed by Google. It enables developers to build **dynamic, collaborative applications** with ease.

Key Features

- **Real-Time Database**: Automatically syncs data across all connected clients.
- **Cloud-Based Storage**: JSON-based storage accessible via REST APIs.
- **Scalable Infrastructure**: Works well with existing services and scales automatically.
- **Authentication & Cloud Messaging**: Secure login and push notifications.

Why Choose Firebase?

- **Instant Backend Setup**: No need to build a separate backend.
- **Fast & Responsive**: Real-time data synchronization.
- **Built-in HTTPS**: Free SSL certificates for custom domains.

Who Uses Firebase?

Companies like **Instacart, 9GAG, and Twitch** rely on Firebase for their backend needs. Firebase is widely adopted, appearing in **1,215 company stacks and 4,651 developer stacks**.

Pros & Cons

Pros

- **Hosted by Google**, ensuring reliability and security.
- **Comes with authentication, messaging, and real-time database services.**
- **Free HTTPS support** for all custom domains.

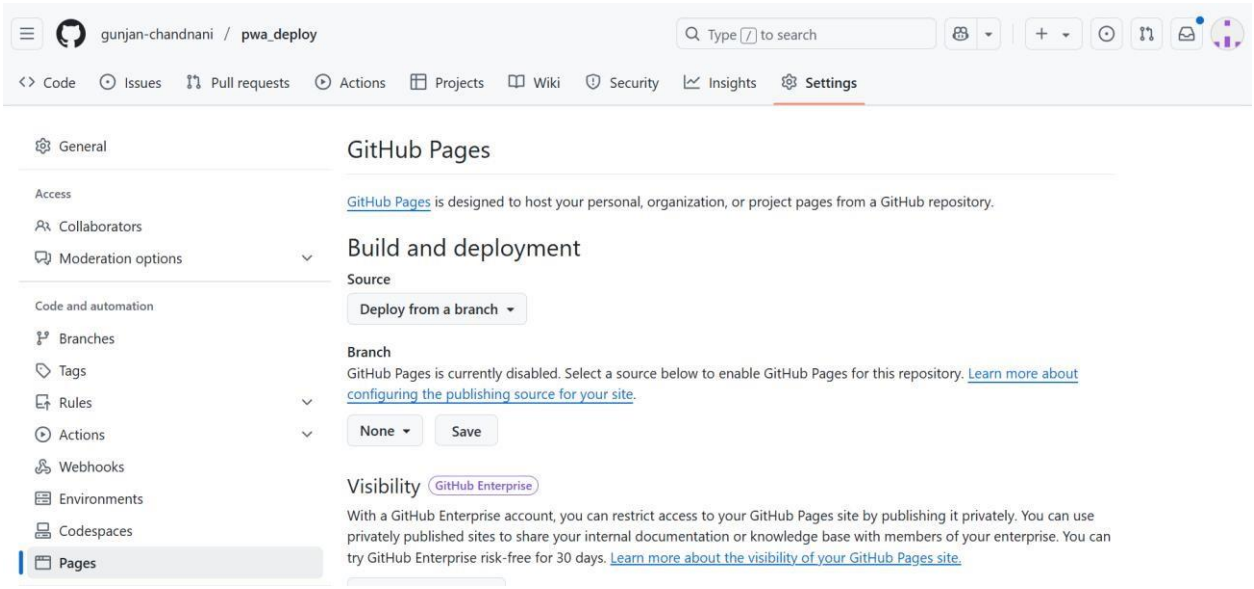
Cons

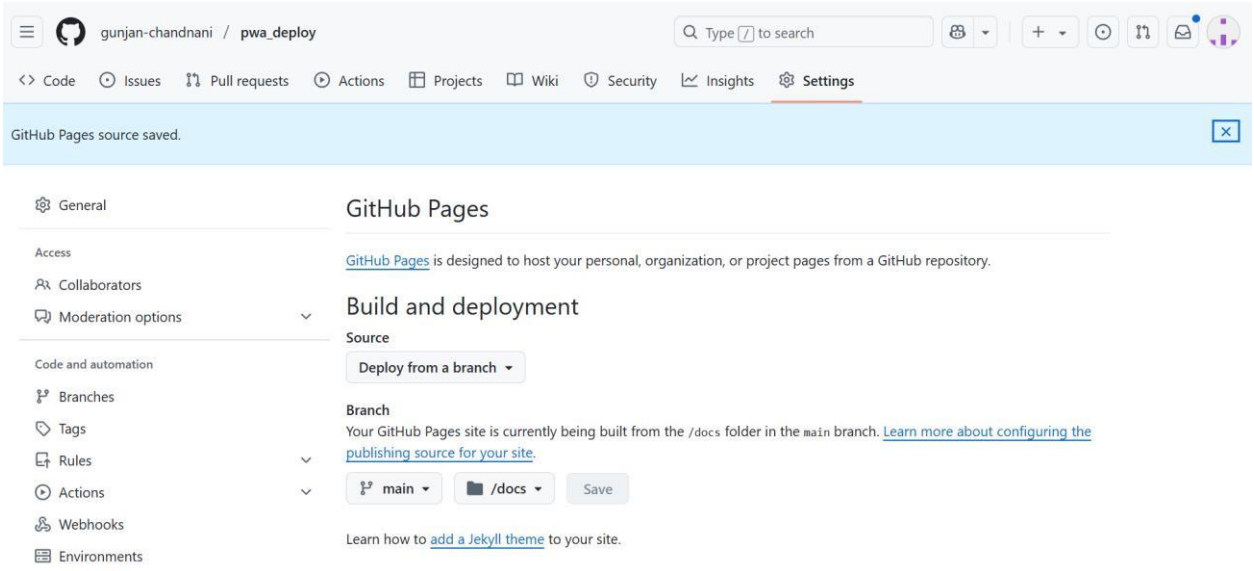
- **Limited Free Plan**: 10 GB of data transfer per month (can be mitigated with a CDN).
- **Command-Line Deployment**: No GUI for hosting.
- **No Built-in Static Site Generator Support**: Unlike GitHub Pages, Firebase doesn't natively support static site generators like Jekyll.

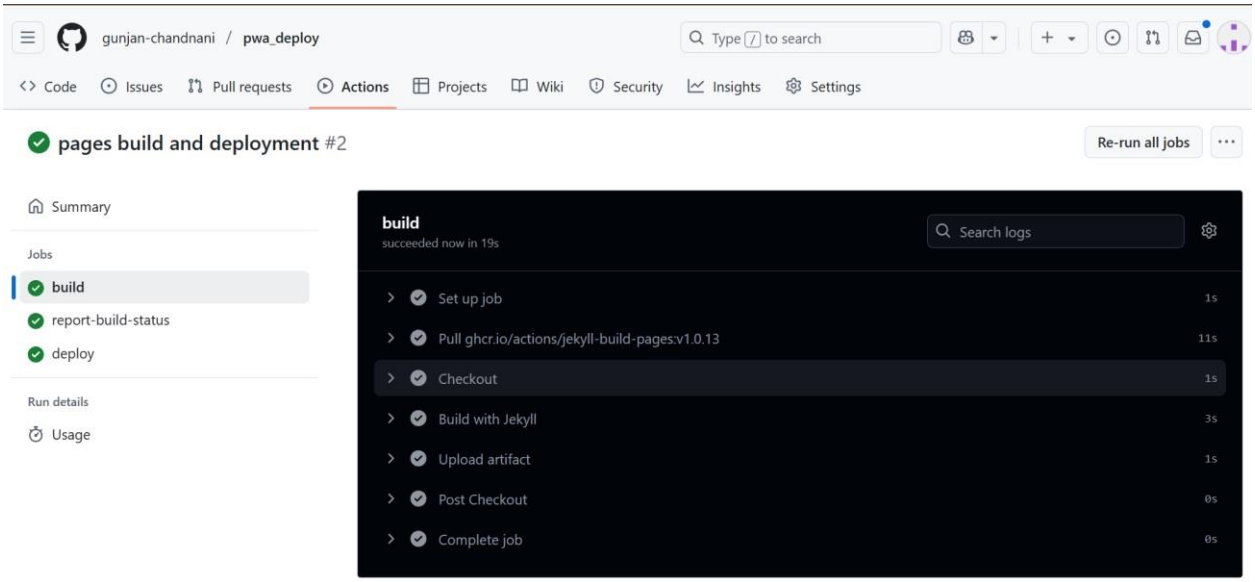
Link to our GitHub repository:

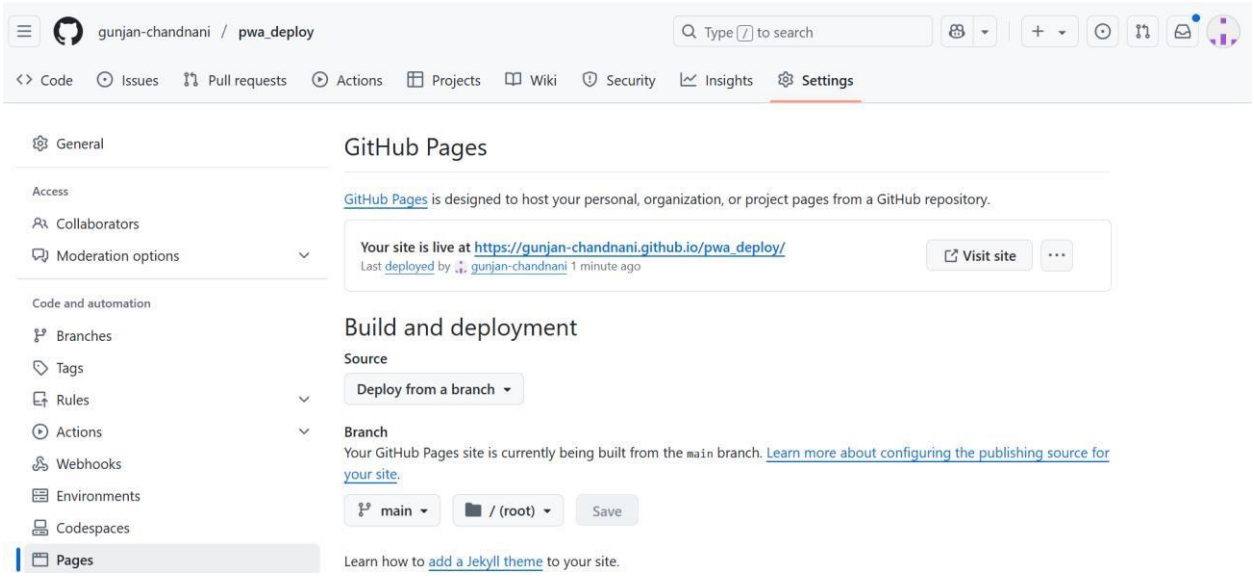
https://gunjan-chandnani.github.io/pwa_deploy/

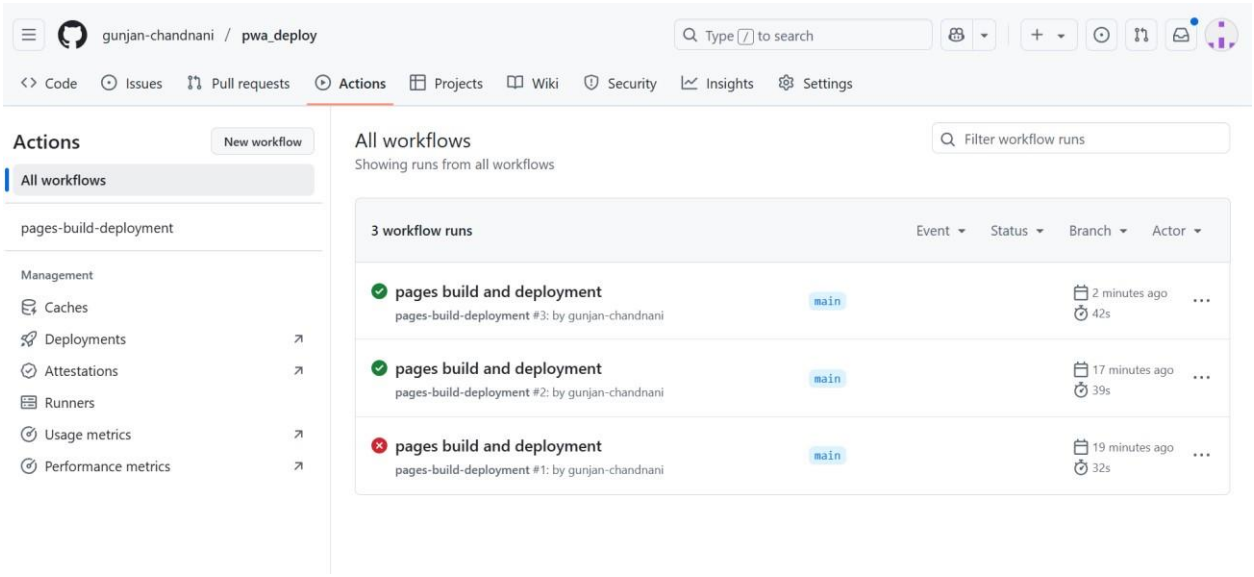
Github Screenshot:

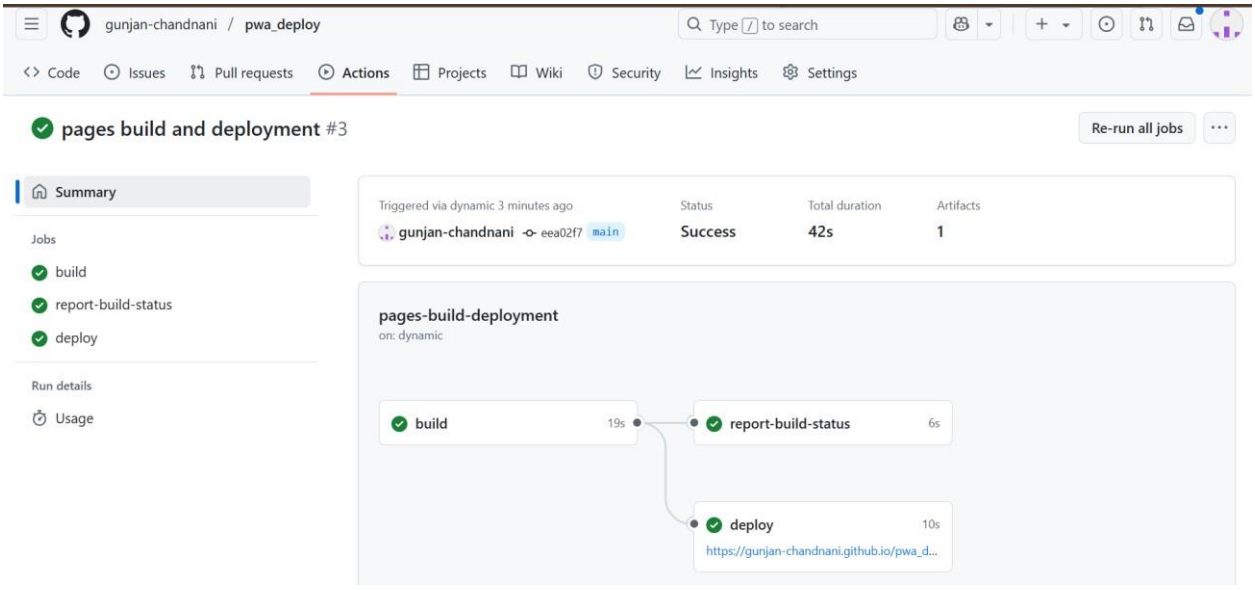



gunjan-chandnani / pwa_deploy
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
GitHub Pages source saved.
General
Access
Collaborators
Moderation options
Code and automation
Branches
Tags
Rules
Actions
Webhooks
Environments
GitHub Pages
GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.
Build and deployment
Source
Deploy from a branch
Branch
Your GitHub Pages site is currently being built from the /docs folder in the main branch. Learn more about configuring the publishing source for your site.
main
/docs
Save
Learn how to add a Jekyll theme to your site.


gunjan-chandnani / pwa_deploy
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
pages build and deployment #2
Re-run all jobs
Summary
Jobs
build
report-build-status
deploy
Run details
Usage
build
succeeded now in 19s
Search logs
Set up job 1s
Pull ghcr.io/actions/jekyll-build-pages:v1.0.13 11s
Checkout 1s
Build with Jekyll 3s
Upload artifact 1s
Post Checkout 0s
Complete job 0s


gunjan-chandnani / pwa_deploy
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
General
Access
Collaborators
Moderation options
Code and automation
Branches
Tags
Rules
Actions
Webhooks
Environments
Codespaces
Pages
GitHub Pages
GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.
Your site is live at https://gunjan-chandnani.github.io/pwa_deploy/
Last deployed by gunjan-chandnani 1 minute ago
Visit site
Build and deployment
Source
Deploy from a branch
Branch
Your GitHub Pages site is currently being built from the main branch. Learn more about configuring the publishing source for your site.
main
/ (root)
Save
Learn how to add a Jekyll theme to your site.


gunjan-chandnani / pwa_deploy
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Actions
New workflow
All workflows
pages-build-deployment
Management
Caches
Deployments
Attestations
Runners
Usage metrics
Performance metrics
All workflows
Showing runs from all workflows
Filter workflow runs
3 workflow runs
Event
Status
Branch
Actor
pages build and deployment
pages-build-deployment #3: by gunjan-chandnani
main
2 minutes ago
42s
pages build and deployment
pages-build-deployment #2: by gunjan-chandnani
main
17 minutes ago
39s
pages build and deployment
pages-build-deployment #1: by gunjan-chandnani
main
19 minutes ago
32s


gunjan-chandnani / pwa_deploy
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
pages build and deployment #3
Re-run all jobs
Summary
Jobs
build
report-build-status
deploy
Run details
Usage
Triggered via dynamic 3 minutes ago
gunjan-chandnani eea02f7 main
Status
Success
Total duration
42s
Artifacts
1
pages-build-deployment
on: dynamic
build
19s
report-build-status
6s
deploy
10s
https://gunjan-chandnani.github.io/pwa_d...

gunjan-chandnani.github.io/pwa_deploy/

FREE SHIPPING ON ALL U.S. ORDERS $50+

Search product

GLOWING

0 $0.00 0

Home Collection Shop Offer Blog

Reveal The Beauty of Skin

Made using clean, non-toxic ingredients, our products are designed for everyone.

Starting at $7.99

Home Collection Shop Offer Blog

More to Discover

Find a Store

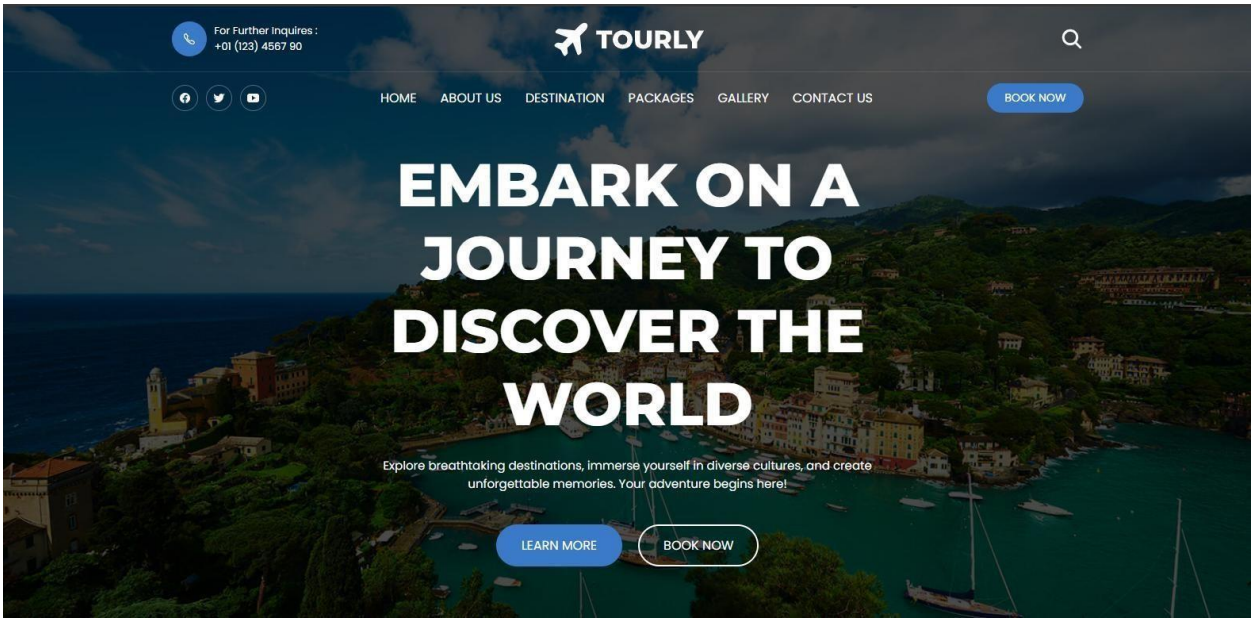
Our Store

From Our Blog

Our Store

Our Story

Our Store


For Further Inquires :
+01 (123) 4567 90
TOURLY
HOME
ABOUT US
DESTINATION
PACKAGES
GALLERY
CONTACT US
BOOK NOW
EMBARK ON A JOURNEY TO DISCOVER THE WORLD
Explore breathtaking destinations, immerse yourself in diverse cultures, and create unforgettable memories. Your adventure begins here!
LEARN MORE
BOOK NOW

MAD & PWA Lab Journal

Experiment No.	11
Experiment Title.	To use google Lighthouse PWA Analysis Tool to test the PWA functioning.
Roll No.	02
Name	Vanshika Ambwani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO6: Develop and Analyze PWA Features and deploy it over app hosting solution
Grade:	

EXPERIMENT NO: - 11

Name:- Vanshika, Gunajn, Akruti **Class:-** D15A **Roll:No: -** 2,6,11

AIM: - To use google Lighthouse PWA Analysis Tool to test the PWA functioning.

Theory: -

Google Lighthouse is an open-source tool designed to evaluate and optimize web applications based on multiple key parameters, including performance, accessibility, Progressive Web App (PWA) implementation, and best practices. It provides a detailed automated audit that helps developers improve their websites efficiently. Unlike traditional manual audits that can take days or weeks, Lighthouse generates insights within minutes with minimal setup.

One of Lighthouse's biggest advantages is its ease of use—simply run it on a webpage or provide a URL, and it will generate a comprehensive report. Lighthouse supports audits for both desktop and mobile versions of a website, ensuring an optimal user experience across different devices.

Key Features and Audit Metrics

1. **Performance**

This metric evaluates how efficiently a webpage loads and becomes interactive. It considers several factors, including:

- Page Load Speed – Measures how quickly content appears to users.
- First Contentful Paint (FCP) – Time taken for the first visible content to load.
- Largest Contentful Paint (LCP) – Measures how long the largest visible content takes to fully render.
- Cumulative Layout Shift (CLS) – Ensures content does not shift unexpectedly, improving user experience.
- Time to Interactive (TTI) – The time taken for the webpage to become fully functional.

Lighthouse assigns a performance score from 0 to 100, where:

- 100 → Top 2% of websites (98th percentile)
- 50 → Around the 75th percentile
- Lower scores → Indicate areas needing optimization

2. **Progressive Web App (PWA) Score**

With the increasing adoption of PWAs, web applications now strive to deliver an app-like experience. Lighthouse evaluates a website's PWA readiness based on Google's Baseline PWA Checklist, which includes:

- Service Worker Implementation – Enables offline functionality and background synchronization.
- App Manifest Compliance – Provides metadata for mobile app-like integration.
- Viewport Configuration – Ensures responsiveness across different screen sizes.
- Performance in Script-Disabled Environments – Verifies that the page functions properly even if JavaScript is disabled.

A high PWA score indicates that the application meets essential criteria for speed, reliability, and mobile usability.

3. Accessibility

This metric determines how well a website supports users with **visual, cognitive, or physical disabilities**. Lighthouse evaluates accessibility based on:

- **ARIA Attributes** – Enhances screen reader support (e.g., aria-required).
- **Text Alternatives for Media** – Ensures images, audio, and video content are accessible.
- **Semantic HTML Usage** – Encourages proper use of elements like <section>, <article>, and <button> for better screen-reader compatibility.

Unlike other metrics, **accessibility follows a pass/fail approach**—missing key features can **significantly impact the overall score**. Improving accessibility ensures **inclusivity for all users**.

4. Best Practices

Lighthouse also assesses whether the website follows **modern web development best practices**, including:

- **Use of HTTPS** – Ensures secure data transmission.
- **Avoiding Deprecated Code** – Prevents the use of outdated elements, directives, and libraries.
- **Secure Password Inputs** – Disables paste-into fields to reduce security risks.
- **User Security Alerts** – Provides notifications for **geo-location access and cookie usage**.

A high **Best Practices score** means the website meets industry standards, leading to better **security, maintainability, and overall performance**.

Manifest.json

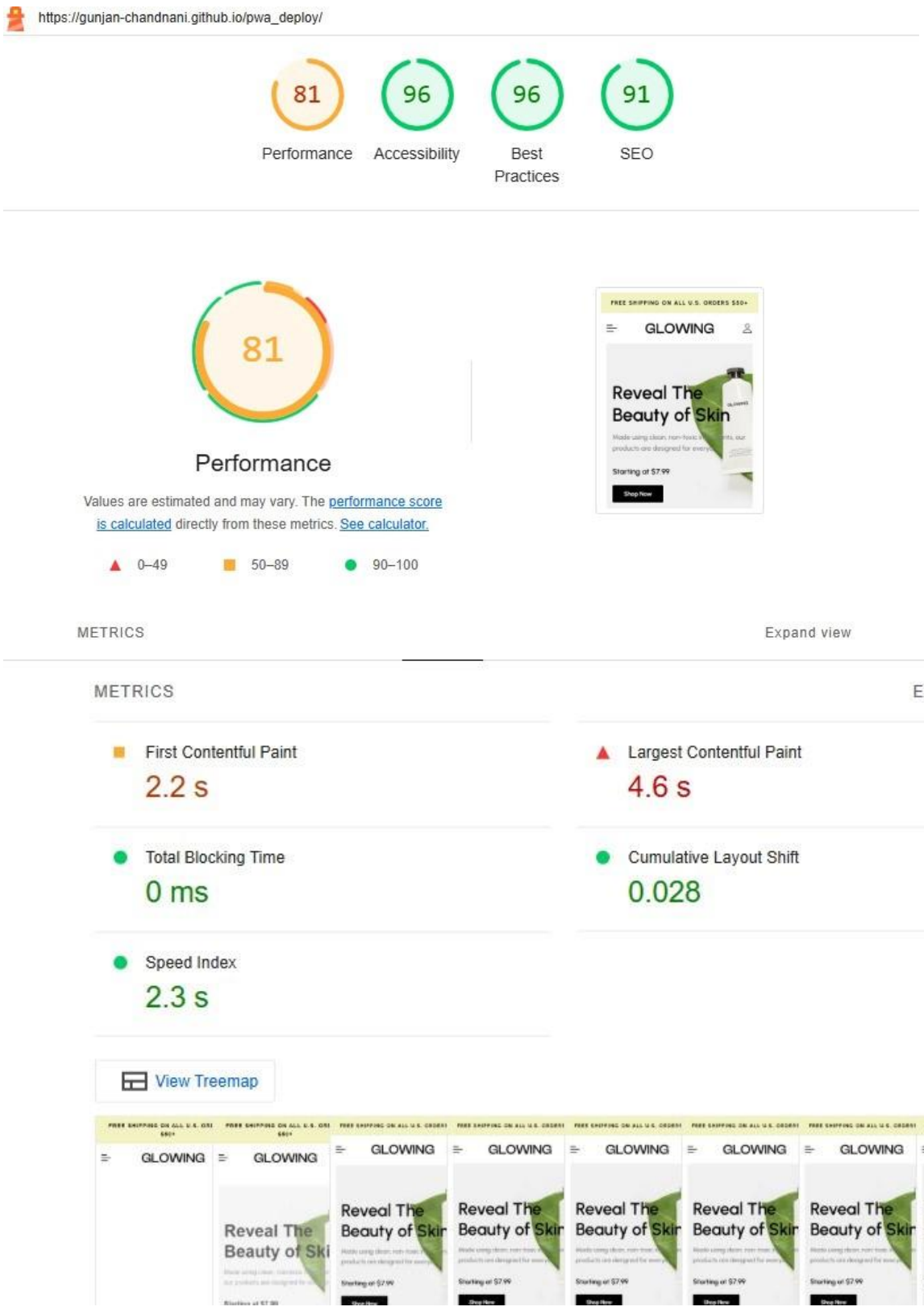
```
{
```

```
"name": "Shopping App",
"short_name": "MyApp",
"description": "An online shopping platform with a wide range of products.",
"start_url": "/",
"scope": "/",
"display": "standalone",
"background_color": "#ffffff",
"theme_color": "#ff6600",
"icons": [
  {
    "src": "assets/images/banner-1.png",
    "sizes": "192x192",
    "type": "image/png",
    "purpose": "any maskable"
  },
  {
    "src": "assets/images/banner-2.png",
    "sizes": "512x512",
    "type": "image/png",
    "purpose": "any maskable"
  }
],
"shortcuts": [
  {
    "name": "Shop Now",
    "short_name": "Shop",
    "description": "Go directly to shopping",
    "url": "/index.html",
    "icons": [
      {
        "src": "assets/images/shortcut-icon.png",
```

```
        "sizes": "96x96",
        "type": "image/png"
      }
    ]
  }
 ]
}
```

Output: -

A] For Mobile



DIAGNOSTICS

- Largest Contentful Paint element — 4,610 ms
- Serve images in next-gen formats — Potential savings of 139 KiB
- Enable text compression — Potential savings of 62 KiB
- Use HTTP/2 — 16 requests not served via HTTP/2
- Defer offscreen images — Potential savings of 36 KiB
- Eliminate render-blocking resources — Potential savings of 170 ms
- Minify CSS — Potential savings of 6 KiB
- Serve static assets with an efficient cache policy — 10 resources found
- Avoid an excessive DOM size — 1,071 elements

https://gunjan-chandnani.github.io/pwa_deploy/

Accessibility

These checks highlight opportunities to improve the accessibility of your web app. Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so manual testing is also encouraged.

ARIA

- Elements use prohibited ARIA attributes

These are opportunities to improve the usage of ARIA in your application which may enhance the experience for users of assistive technology, like a screen reader.

ADDITIONAL ITEMS TO MANUALLY CHECK (10) Show

These items address areas which an automated testing tool cannot cover. Learn more in our guide on conducting an accessibility

ADDITIONAL ITEMS TO MANUALLY CHECK (10) Show

These items address areas which an automated testing tool cannot cover. Learn more in our guide on conducting an accessibility review.

PASSED AUDITS (24) Show

NOT APPLICABLE (32) Show

Best Practices

B]For Desktop

https://gunjan-chandnani.github.io/pwa_deploy/

Performance Accessibility Best Practices SEO

Performance

Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.

▲ 0–49 ■ 50–89 ● 90–100

METRICS Expand view

97 96 96 91

- First Contentful Paint
  0.7 s
- Largest Contentful Paint
  1.2 s
- Total Blocking Time
  0 ms
- Cumulative Layout Shift
  0.001
- Speed Index
  0.9 s

View Treemap

Show audits relevant to: All FCP LCP TBT CLS